혼자서 만드는 가상화폐 자동거래 시스템
https://wikidocs.net/book/1436

지지은이 : 박재현 **개인 이메일은 hyunny82@naver.com이다**
최최종 편집일시 : 2018년 11월 6일 9:37 오후

파이썬으로 만드는 암호화폐 자동 거래 시스템
http://www.yes24.com/24/goods/60203451?scode=032&OzSrank=1

많은 관심바랍니다. 관련 문의사항은 언제든지 메일로 연락주세요.
정오표: http://cafe.naver.com/myautotrading/4
소스코드: https://gitlab.com/hyunny88/auto-trading

경제적 자유를 꿈꾸는 모든 사람들을 위해 자신만의 시스템을 구축할 수 있도록 가이드 북
이 책을 통해 시스템을 구성하는데 필요로 하는 기본적인 요소들을 익힐 수 있도록 하는게 목표이다.

Linux 관련, Git, Gitlab, REST API, MongoDB, Slack, Celery, Redis, Flask, AngualrJS, ReactJS, Ionic, Nginx, Supervisord 등을 다 사용하여 전체 프로젝트를 완성할 것이다.

각 키워드만 해도 다 책 한 권씩 쓸 수 있을 정도로 매우 많은 내용이 있다. 현실적으로 필자 직업이 글을 쓰면서 먹고 사는 직업이 아니라 처음부터 상세하게 쓸 수는 없을 듯 하지만 지속적으로 보완하면서 책을 완성해가고자 한다. 우선은 이 프로젝트를 진행해보면서 가볍게 사용해본 뒤에 실전에서 필요한 부분이 생기면 그때 깊게 고민하고, 공부해서 자기 것으로 만들었으면 하는 바람이다.

그리고 여러 기술스택위에서 시스템 아키텍쳐를 어떻게 가져갈지 한번 고민해볼 수 있는 시간이 되었으면 한다.
덤으로 부가수익까지 올릴 수 있다면 금상첨화.

현재 5개월 정도 운영하고 있는데 40%(실제 수익은 40%이나 자동 트레이딩으로 얻은 수익은 35%정도 이다.) 정도의 수익을 낸 상황이다. **현재 운영하고 있는 방식도 공개할 것이다.**

굉장히 설명이 빈약할 수 있다. 글을 쓰는 작업이 익숙하지 않은 점도 있지만, 주어진 시간도 많지 않아 우선 전체 목차를 1차적으로 완성한 이후에 내용을 추가적으로 보충할 예정이다. 혹시 여기 목차 중에 자세히 알고 싶은 부분에 대한 요청이 있으면 우선적으로 작성할 수 있

도록 하겠다. **개인 이메일은 hyunny82@naver.com이다** 모든 진행 과정은 아래 gitlab에 공개하도록 하겠다. 소스는 책이 써가면서 해당 내용을 틈틈히 update하도록 하겠다. https://gitlab.com/hyunny88/auto-trading.git

책을 한번 읽다보면 알겠지만... 투자이든 투기든... 코인으로 돈벌려는 목적보다는 자기만의 시스템을 구축하는데 초점이 맞춰져서 진행된다. **자동거래도 충분히 손해를 볼 수 있다는 점 인지하길 바란다.**

01. 코인 자동거래 시스템 프로젝트

2017년에 가장 이슈였던 단어를 하나 꼽자면 바로 비트코인일 것입니다. 그리고 대부분의 직장인이 그렇듯 재테크에 관심 없는 직장인은 없을 것입니다. 저는 **재테크와 비트코인**(암호화폐)이라는 두 개의 키워드로 경제적 자유를 꿈꾸는 모든 분들을 위해 자신만의 트레이딩 로직으로 움직이는 시스템을 구축할 수 있도록 안내하는 책을 쓰고 싶어서 이 글을 쓰기 시작했습니다. 이 책의 목표는 암호화폐 트레이딩 시스템을 구성하는 데 필요한 기본적인 요소를 익히는 것입니다. 저는 개인적으로 다양한 기술에 관심이 많습니다. 처음에는 나만의 시스템을 만들고 싶었습니다. 그러던 중 암호화폐 관련 자동화 시스템이 여러 측면에서 시스템으로 구현하기에 적절하다고 판단해서 만들게 됐습니다. 그리고 시스템을 하나씩 구현하고 마무리될 때쯤 무엇이라도 남기지 않으면 잊히는 프로젝트가 될 것 같아 인터넷에 프로젝트 진행 과정을 공개했습니다. 여기까지 진행해보니 책으로 엮어서 많은 사람이 조금 더 접근하기 쉽게 만들고 싶었습니다. 처음부터 책을 쓰겠다는 생각으로 여기까지 온 것이 아니라 하나하나 작은 목표를 두고 시작한 프로젝트가 책 한 권을 쓸 수 있을 정도까지 진행됐습니다. 저에게 암호화폐는 투자보다는 나 자신을 트레이닝하고 조금 더 레벨업할 수 있는 발판으로 작용했습니다.

여러분이 이 책을 보고 있는 이유는 암호화폐 시장에 관심이 있기 때문일 것입니다. 많은 매체를 통해 정보를 접했겠지만 암호화폐 시장은 이제 막 태동 단계로 혼란의 연속입니다. 각 나라의 규제나 사소한 뉴스만으로도 하루에 가격이 20 ~ 30% 출렁입니다. 심하면 50%까지도 변동이 일어납니다. 만약 암호화폐에 자산을 투자한 사람이면 온종일 암호화폐의 가격을 보고 있을 것입니다. 일확천금을 꿈꾸면서 자신의 삶이 피폐해져 가는지 깨닫는 데는 그리 오랜 시간이 걸리지 않을 것입니다. 이 책에서는 이런 혼란의 시장에서 시스템 트레이딩을 통해 위험성은 최대한 피하면서 시스템으로 24시간 대응 가능한 시스템을 만들려고 합니다. 즉, 우리의 목적은 일확천금을 노리는 것이 아니라 혼돈의 시장에서도 지속 가능한 성장을 위해서라는 것을 다짐하고 진행했으면 합니다.

마지막 편집일시 : 2018년 5월 3일 9:57

01.1. 프로젝트의 진행 방향 설정

프로젝트를 진행하기에 앞서 전체적인 책의 진행 방향을 알아보겠습니다. 이 책에서는 세 가지 주요 내용을 다루고자 합니다. 1) 거래소가 제공하는 API를 둘러보고 파이썬으로 트레이딩 가능한 모듈을 만들어 봅니다. 2) 암호화폐 가격 데이터를 저장, 가공, 분석하는 방법을 알아

봅니다. 3) 자신만의 트레이딩 로직을 구현해 자동으로 동작하는 시스템을 만들어 봅니다. 이 책에서는 트레이딩 알고리즘과 관련된 내용은 다루지 않습니다. 이 프로젝트를 실행할 수 있도록 최소한의 로직만 구현합니다. 트레이딩 알고리즘은 독자 본인이 설계하셔서 자신만의 알고리즘을 구현하길 바랍니다. 1장에서는 리눅스, 파이썬 개발 환경을 구성합니다. 이어서 데이터 분석을 위한 주피터를 설치해 사용해 봅니다. 그리고 프로젝트의 소스코드 관리를 어떻게 할지 알아봅니다. 마지막으로 이 책에서 만든 프로젝트의 전체 구성도를 그려보고 앞으로 만들어나갈 기능을 정리하겠습니다.

마지막 편집일시 : 2018년 5월 3일 9:57

01.2. 개발 환경 구성

이 책의 프로젝트는 리눅스에서 개발을 진행합니다. 이번 절에서는 윈도우, 맥 사용자가 리눅스 개발 환경을 구성하는 데 필요한 내용을 설명합니다. 이미 리눅스를 사용 중인 독자라면 이번 절을 넘어가도 무방합니다. 윈도우와 맥 사용자는 현재 사용 중인 운영체제에 버추얼박스(VirtualBox)를 설치하고 가상 머신에 리눅스 운영체제를 설치해 개발 환경을 구성합니다. 이 책에서는 윈도우 환경을 기준으로 버추얼박스 설치 및 개발 환경 구성과 관련된 내용을 알아보겠습니다.

마지막 편집일시 : 2018년 5월 3일 9:57

1. 01.2.1. 버추얼박스 설치

버추얼박스는 오라클에서 개발 중인 가상화 소프트웨어입니다. 윈도우, 맥 OS X, 리눅스 등 다양한 플랫폼을 지원하며, 무료로 내려받아 사용할 수 있습니다. 먼저 https://www.virtualbox.org/ 에서 버추얼박스를 내려받습니다.

그림 1.1 버추얼박스 다운로드 페이지(https://www.virtualbox.org/wiki/Downloads)

이 글을 쓰는 시점에 버추얼박스의 최신 버전은 5.2입니다. [Download] 메뉴를 눌러 이동한 후 [VirtualBox 5.2.6 platform packages]에 있는 [Windows hosts] 링크를 클릭해 버추얼박스를 내려받습니다. 추가로 바로 밑에 있는 [VirtualBox 5.2.6 Oracle VM VirtualBox Extension Pack]의 [All supported platforms]를 클릭해 확장팩도 함께 내려받습니다. **확장팩은 버추얼박스에서 USB, Display, RDP 등 여러 가지 기능을 사용**하는 데 필요합니다.

내려받은 파일을 실행하면 그림 1.2와 같은 화면이 나타나면서 설치 과정이 시작됩니다. [Next] 버튼을 눌러 다음 단계로 넘어갑니다.

그림 1.2 버추얼박스 설치 옵션

메뉴 추가에 관련된 항목을 설정합니다. [Next]를 눌러 다음 단계로 넘어갑니다.
그림 1.3 버추얼박스 메뉴 추가 설정

"Network Interfaces" 관련 경고문이 나타나면 설치를 진행할지 물어봅니다. [Yes]를 선택합니다.
그림 1.4 네트워크 구성 경고

설치 완료 후 [Finish] 버튼을 눌러 설치를 마치고 버추얼박스를 실행해 봅니다.
그림 1.5 버추얼박스 설치 완료

다음과 같이 환영 메시지가 나타나면 정상적으로 잘 설치된 것입니다.
그림 1.6 버추얼박스 관리자
다음 절에서는 **버추얼박스에 새로운 리눅스 머신을 생성해 보겠습니다.**

마지막 편집일시 : 2018년 5월 3일 10:02

01.2.2. 리눅스 설치

이번 절에서는 버추얼박스에 리눅스 머신을 생성해 보겠습니다.
프로젝트에서 사용할 리눅스 운영체제는 **우분투 16.04 LTS**입니다. 우분투 설치 이미지는 https://www.ubuntu.com/download 에서 받을 수 있습니다. 반드시 **Desktop 버전을 내려받습니다.** 버전은 16.04를 선택합니다(17.04은 지원하지 않는 패키지가 있을 수 있습니다).
그림 1.7 우분투 데스크톱 다운로드

[Download] 버튼을 누르면 페이팔(PayPal) 기부를 물어보는데 기부할 의사가 없다면 [Not now, take me to the download] 를 누릅니다. 내려받기가 끝나면 **용량이 1.5GB 정도**인 **ubuntu-16.04.3-desktop-amd64.iso** 파일을 확인할 수 있습니다. 윈도우를 설치할 때 CD나 USB를 이용해 PC에 설치하듯 버추얼박스에도 이 iso 파일로 설치를 진행하게 됩니다. 다시 버추얼박스로 돌아와서 [새로 만들기] 버튼을 누릅니다.

그림 1.8 가상 머신의 이름 및 종류 설정

이름을 입력한 후 종류[Linux], 버전**[Ubuntu (64-bit)]**를 선택한 후 [다음] 버튼을 누릅니다.
그림 1.9 가상 머신의 메모리 설정

메모리는 개인 PC의 메모리 용량에 따라 다릅니다. 호스트 운영체제(윈도우)의 50% 정도를 할당합니다. 즉, 8GB 메모리를 사용 중이면 4GB를 할당하고 [다음]을 클릭합니다. 다음으로

하드 디스크 공간을 할당합니다. [지금 새 가상 하드 디스크 만들기]를 선택합니다.
그림 1.10 가상 머신의 하드 디스크 만들기

가상 하드 디스크 파일의 종류로 [VDI (VirtualBox 디스크 이미지)]를 선택합니다.
그림 1.11 가상 머신의 하드 디스크 종류 설정

물리적 하드 드라이브에 저장하는 방식을 선택하는 화면이 나왔을 때 **하드 디스크에 여유가 있다면 [고정 크기]를** 선택합니다. 하드 디스크에 여유가 없다면 [동적 할당]을 선택합니다.
그림 1.12 가상 머신의 하드 디스크 할당 방식 설정

파일 위치 및 크기에서는 개인의 하드 디스크 여유 공간에 따라 적당량을 할당합니다. 최소 10GB는 할당해야 합니다. 50GB 정도면 넉넉히 사용할 수 있습니다.
그림 1.13 가상 머신의 하드 디스크 용량 설정

지금까지 가상 머신에 대한 설정을 진행했습니다. 설정을 완료하고 나면 **[시작]** 버튼을 누릅니다. 초기 화면에 시동 디스크 입력을 물어봅니다. 여기서 우분투 사이트에서 내려받은 **ubuntu-16.04.3-desktop-amd64.iso** 파일을 선택합니다.
그림 1.14 가상 머신 시작 화면

이제 우분투 리눅스 설치가 시작됩니다. 한국어를 선택한 후 [Ubuntu 설치]를 누릅니다.
그림 1.15 우분투 언어 설정 및 설치 시작

[Ubuntu 설치 중 업데이트 다운로드]를 선택합니다.
그림 1.16 설치 준비 옵션 선택

설치 형식에서는 [디스크를 지우고 Ubuntu 설치] 를 선택합니다. 이때 디스크는 이전에 하드 디스크로 50GB를 할당한 디스크를 말합니다. **파티션을 나누고 싶거나 리눅스 영역을 나누고 싶다면 [기타]를 선택합니다.** 프로젝트는 디스크를 지우고 설치하는 방식으로 진행하겠습니다.
그림 1.17 디스크 설치 옵션

[지금 설치]를 누르고 위치와 키보드 방식, 기본 유저 생성에 관한 정보를 입력하면 리눅스 설치가 진행됩니다. 설치가 완료되고 [재시작] 버튼을 누르면 우분투 화면을 만날 수 있습니다.
그림1.18 우분투 시작 화면

보다시피 리눅스 설치가 과거에 비교해 매우 쉬워졌습니다. 윈도우를 설치하는 것과 크게 다르지 않습니다. 설치부터 부팅 후 로그인까지 윈도우나 맥 OS X와 유사한 사용자 경험을 제공합니다. 그리고 사용자가 부족함 없이 쓸 수 있도록 많은 소프트웨어를 제공하고 있으므로 리눅스에 대한 거부감이 줄어들었으면 합니다. 이번 절에서는 리눅스 머신까지 구성했습니다. 이어지는 절에서는 파이썬과 관련된 개발 환경을 구성하겠습니다.

마지막 편집일시 : 2018년 5월 3일 10:03

==========

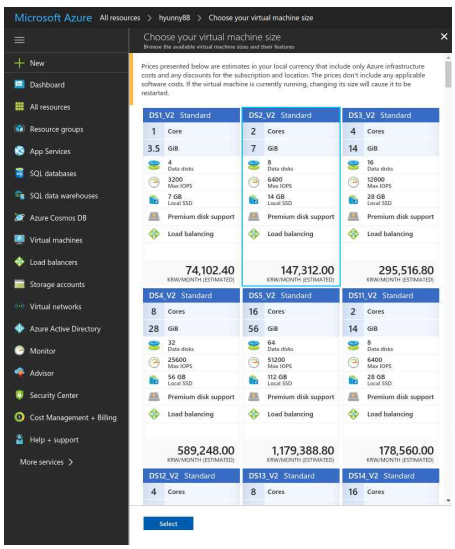
02. 개발 서버 및 개발 환경 구축하기

우리가 진행하는 프로젝트는 개인 PC에서 개발하지만, 최종 완성본을 PC에서 구동할 수는 없다. 안되는 건 아니지만 **안정적인 시스템 운영을 위하여 운영 서버는 별도로 구축하여 개발된 프로젝트를 구동하도록 한다.** 그리고 **소스코드 형상관리도 해야 한다.** 리눅스 서버 구성, 개발 환경 설정, 소스코드 형상관리 등 각각의 주제들로만 해도 책을 하나씩 쓸 수 있을 정도로 양이 많다. 우리의 목표는 위 내용을 익히는 게 아니기 때문에 필요한 최소한의 내용만 여기서는 기술하며 부가적으로 필요한 내용은 Google에서 참조 바란다. 추가 설명 요청이 있는 경우에만 부족한 내용을 추가할 수 있도록 하겠다. Linux server, SSH, Python 개발 환경, 개발 툴, 형상관리에 대한 내용을 다 적을 예정이지만 이 부분은 3가지 목표사항을 다 정리한 뒤 보충할 수 있도록 하겠다.

마지막 편집일시 : 2017년 10월 8일 1:22

2. 02.1. Linux Server

개발환경 및 운영할 서버는 Linux Ubuntu 16.04에서 진행한다. 필자는 운영하는 서버는 **Azure에 Ubuntu 16.04 Server** 이다. 실제 개발은 개인 PC에 Ubuntu 16.04 Desktop을 설치해서 진행한다.



Microsoft 사의 Azure Cloud Platform 가격 책정 표

Azure 기준으로 1core에 3.5G 기준 7만4천원 정도 책정되어있다. 필자는 DS2_V2인 2core에 7G 환경에서 현재 프로젝트를 운영 중이다. Linux 호스팅 업체는 본인의 기준에서 선택하면 된다.

02.2. 개발 Linux 환경 구성하기

일반적으로 윈도우나 Mac 환경에서 리눅스를 사용하고 싶다면 VirtualBox를 사용할 수 있지만 팀으로 개발을 진행하는 경우라면 동일한 개발환경 가져가면서 개발하기는 쉽지 않다.

Vagrant는 이런 부분을 도와주는 가상화 툴이다. Vagrant는 Ruby로 개발되어져 있어 일종의 Configuration 파일인 **Vagrantfile을 ruby로 작성**해야 되지만 어렵지 않게 사용할 수 있으니 걱정하지 말자. 그리고 공식문서가 잘 정리되어있다. 여기서는 ubuntu16.04 환경을 구성하는 것 까지만 해보겠다.

마지막 편집일시 : 2017년 12월 31일 8:48

02.2.1. Vagrant

02.2.1.1. Install

Vagrant 설치 전에 VirtualBox를 설치해야 한다. 공식사이트에서 5.x 버전을 설치하자. Guest Additions도 추가로 설치해야한다. 글을 작성하는 시점에 버전은 5.2.4이다. 그리고 Vagrant도 공식사이트에서 받아서 설치한다. 버전은 2.0.1 이다.

마지막 편집일시 : 2017년 12월 31일 8:49

02.2.1.2. Vagrant up

vagrant까지 설치가 완료되었다면 **C:\vagrant\ubuntu16.04 디렉토리를 만든 후**에

vagrant init bento/ubuntu-16.04

vagrant up

로그인 후 uname 및 배포판 버전을 확인해보자.

vagrant ssh

uname -a

cat /etc/*release

16.04 LTS 가 정상적으로 설치된 것을 확인할 수 있다.

마지막 편집일시 : 2017년 12월 31일 8:51

02.2.1.3. Configuration

Vagrant init을 하면 해당 디렉토리 밑에 Vagrantfile이 생기게 된다. 이 Vagrantfile에 여러가지 개발환경 관련 셋업을 설정할 수 있다. 그리고 **이 Vagrantfile을 개발팀이 동일하게 가져가면 동일한 개발환경에서 개발을 진행할 수 있다.**

Vagrant는 굉장히 많은 기능들이 있다. Network 구성, 공유폴더, Multi-Machine, 서버에 shell환경에서 설치를 지원하는 provisioning, Vagrant Share(환경 공유) 다양한 기능에 대해서는 공식사이트의 docs를 참고하자. 이 모든 내용을 다루는 것은 이 책의 범위를 넘어가기 때문에 따로 적지는 않겠다.

마지막 편집일시 : 2017년 12월 31일 8:51

==========

02.3. SSH

개발 환경에서 서버로 접속하는 것은 SSH를 사용한다. **SSH는 공개키 기반으로 나의 공개키는 서버에 입력해두고 비공개키는 나의 PC에 저장되어 있다**. 이 두개 Key의 조합으로 SSH 연결이 된다. 일반적인 telnet application을 사용하면 사용자가 입력하는 모든 내용이 평문으로 전송된다. WireShark 같은 network packet 분석 tool로 보면 모든 내용은 훤히 들여다 볼 수 있다. 그러나 SSH로 연결되면 모든 내용이 암호화 되기 때문에 일반적인 방법으로 알아볼 수 없다. 이런 보안적인 이슈를 해결하고자 나온 방식이 SSH이며 **remote로 우리가 서버에 접속할때는 ssh를 이용하여 접속하게 된다.**

마지막 편집일시 : 2017년 12월 31일 8:5

02.3.1. Key생성

$ **ssh-keygen**을 실행해보자. 아무런 option을 주지 않으면 기본 암호 알고리즘은 **rsa**로 생성된다. 그리고 key를 저장할 위치와 passphrase를 입력하라고 한다. passphrase는 일종의 비밀번호로 나의 비공개키를 입력한 값으로 암호화한다. 빈값으로 설정해도 상관없지만 보안상 문제가 될 수 있다.

자세한 내용은 (https://www.ssh.com/ssh/passphrase#sec-What-is-Passphrase) 사이트를 참고하고 최소 15자에서 20자로 추측하기 어려운 문자를 권고하고 있다.

이렇게 입력을 끝내고 나면 **id_rsa, id_rsa,pub** 두개 파일이 생성되어 있는 것을 확인할 수 있다. .pub 파일은 공개키 파일로 해당 내용은 서버에 등록하면 된다. **id_rsa파일은 본인의 비밀키로 유출되면 보안상 문제가 발생한다.**

마지막 편집일시 : 2017년 12월 31일 8:52

02.3.2. Key 등록

서버의 HOME/.ssh/directory밑에 authorizedkeys 파일을 하나 만들자.

그리고 idrsa.pub 파일의 내용을 authorizedkeys에 copy하자.

ssh-copy-id user@host를 이용해도 동일하다.

그리고 파일의 접근권한을 변경해야 한다.

$ chmod 700 ~/.ssh

$ chmod 600 ~/.ssh/id_rsa

$ chmod 644 ~/.ssh/id_rsa.pub

$ chmod 644 ~/.ssh/authorized_keys

$ chmod 644 ~/.ssh/known_hosts

마지막 편집일시 : 2017년 12월 31일 8:52

02.3.3. 연결

아래와 같이 입력 후 서버에 비밀번호 로그인 없이 접속할 수 있다면 성공한 것이다.

$ ssh user@host

만약 sign_and_send_pubkey: signing failed: agent refused operation 와 같이 fail msg를 내면서 비밀번호를 물어본다면 내 PC에서 아래 커맨드로 agent를 수행하자.

$ **eval "$(ssh-agent -s)"**
ssh의 자세한 key 교환이나 접속 과정을 보고 싶다면 ssh -vvv user@host로 모든 과정을 눈으로 확인할 수 있다.

마지막 편집일시 : 2017년 12월 31일 8:53

02.4. Python 개발 환경 구성하기

Python은 3.5+를 기본으로 개발한다. 특별히 우리가 이번에 진행하는 프로젝트에서는 2.x, 3.x를 구분될 필요는 없어 3.x를 사용할 것 이다. **python을 사용하게 되면 수많은 library를 안쓸 수 없다**. 하나의 python으로 여러 프로젝트를 진행하게 되면 각 프로젝트에서 진행할 때 사용한 **library에 대한 관리를 하기가 어렵다. 이러한 부분을 관리할 수 있게 도와주는 tool이 virtualenv, virtualenvwrapper 이다.**
이번 장에서는 virtualenv, virtualenvwrapper의 사용방법을 익혀보자.

마지막 편집일시 : 2017년 12월 31일 8:56

02.4.1. Virtualenv, Virtualenvwrapper

python을 사용하게 되면 수많은 library를 안쓸 수 없다. 하나의 python으로 여러 프로젝트를 진행하게 되면 각 프로젝트에서 진행할 때 사용한 **library에 대한 관리를 하기가 어렵다**. 이러한 부분을 관리할 수 있게 도와주는 tool이 virtualenv, virtualenvwrapper 이다. 이번 장에서는 virtualenv, virtualenvwrapper의 사용방법을 익혀보자.

마지막 편집일시 : 2017년 12월 31일 8:56

02.4.1.1. Install

Virtualenv: Python 가상환경 지원 tool

https://virtualenv.pypa.io/en/stable/

Virttualenvwrapper: Virtualenv의 사용을 편리하게 하기 위한 extension tool

https://virtualenvwrapper.readthedocs.io/en/latest/

```
$ sudo pip install virtualenv
$ sudo pip install virtualenvwrapper
$ export WORKON_HOME=~/.virtualenvs
$ mkdir -p $WORKON_HOME
$ source /usr/local/bin/virtualenvwrapper.sh
$ mkvirtualenv auto-trading —python=python3
```

mkvirtualenv 에서 auto-trading 이라는 이름으로 가상환경을 만들고 여기서는 python3를 기본 python으로 지정하였다. 실제 이 command를 치게되면 python3를 .virtualenvs

아래에 auto-trading 디렉토리 및에 copy되어 설치하게 되고 path등이 .virtaulenvs/auto-trading/bin을 우선적으로 참조하게 되어진다.

마지막 편집일시 : 2017년 12월 31일 8:5

02.4.1.2. Usage

2.3.1.1에서 mkvirtualenv 로 가상환경까지 만들었다면

mkvirtualenv -> 가상환경 생성 rmvirtuaelnv -> 가상환경 삭제 workon [이름] -> 이름에 해당하는 가상환경으로 변환

아래 와 같이 pip로 설치한 항목을 requiremetns.txt로 떨어트릴 수 있다.

$ pip freeze >> requirements.txt

반대로 requirements.txt에 있는 항목을 pip로 설치할때는 -r 옵션을 사용하여 프로젝트에서 사용한 library를 한번에 설치할 수 있다.

$ pip install -r requirements.txt

마지막 편집일시 : 2017년 12월 31일 9:0

==========

02.5. 개발툴

개발툴은 Vim, Visual code를 이용한다. 대표적인 Python 개발 툴은 pydev, pycharm 등 좋은 IDE가 많다. 필자도 여러가지를 다 사용하고 있지만, 주로 Visual studio code에 Vim plugin을 셋팅해서 사용하고 있다.

마지막 편집일시 : 2017년 12월 31일 8:53

3. 02.6. 최종 구성도

최종적으로 완성되면 다음와 같은 구성도가 된다. 안정적인 시스템 운영을 위해서는 HA 구성이 필수이지만, 이번 프로젝트는 혼자서 만들어 사용하는 시스템이기 때문에 HA 구성과 관련된 내용은 제외한다.

녀애마지막 편집일시 : 2017년 10월 8일

02.6. 형상관리

Source code의 형상관리는 git으로 하며 repository는 gitlab를 이용한다. gitlab을 사용하게 된 결정적인 이유는 무료로 private 프로젝트를 지원하기 때문이다. github도 private 프로젝트를 지원하지만 유료 서비스이다.

마지막 편집일시 : 2017년 12월 31일 8:53

02.6.1. Git

gitlab을 사용하기 이전에 git을 우선 설치하자. **sudo apt-get install git**

git과 관련된 내용은 추후 작성 예정.

마지막 편집일시 : 2017년 12월 31일 8:53

02.6.2. Gitlab

GItlab에 서비스를 이용하기 위해서 회원가입을 하자 https://about.gitlab.com/ gitlab과 관련된 내용은 추후 작성 예정.

마지막 편집일시 : 2017년 12월 31일 8:54

4. 02.7. 프로젝트 디렉토리 구성 및 initail commit

프로젝트 폴더 구성 및 git, gitlab 연동까지 해보자.

gitlab login 후 New project로 auto-trading 을 만들고 내 pc로 clone 해오자. Create a new repository의 명령어를 그대로 수행하자. 이 글에서 작성하는 모든 내용은 아래 repository에 공개한다. https://gitlab.com/hyunny88/auto-trading.git

마지막 편집일시 : 2017년 10월 8일 1:24

==========

03. 거래소 모듈 구현

우리나라에 현재 대표적인 거래소는 빗썸, 코인원, 코빗이 있다. 그리고 신규거래소가 오픈 예정이다. API를 통한 거래를 지원하는 곳도 있고 하지 않는 곳도 있지만 코빗이 API에 관련된 거래를 비교적 잘 지원하고 있다. 그리고 추가로 CoinOne API도 추가로 구현하여 두개 모듈을 공통적으로 사용할 수 있도록 할 예정이다.

마지막 편집일시 : 2017년 10월 26일 1:21

03.1. Korbit

최근 몇 년 동안 가상화폐 시장이 폭발적으로 증가해왔다. 거품 논란도 많지만 어찌되었든 현재 가상화폐 시장은 증권거래 시장과 맞먹을 정도의 규모로 성장해왔고 미국, 일본 등에서는

통화로서의 기능도 하고 있다. 달러, 엔화, 위안화 등 모든 통화들이 마찬가지이지만 가격이 고정되어있지 않으며 여러 가지 요인으로 등락을 거듭한다.

필자는 투자 전문은 아니기 때문에 여러 논란에 대해서 답할 수는 없지만, 시스템적으로 자동화가 기존 증권사보다 간단하며 등락 폭이 기존 통화보다 크기 때문에 자동화를 통한 수익도 어느 정도 낼 수 있다고 판단되어 시스템을 만들게 되었다. 국내에 가상화폐 거래소는 10여 군데가 있지만 그중에서 Korbit을 거래소를 기준으로 시스템을 구축하고자 한다. 이후에 다른 거래소를 추가할 수 있도록 설계에 반영하고자 한다. **Korbit은 웹상으로 모든 거래를 지원하며 API를 통한 거래도 지원하고 있다.**

마지막 편집일시 : 2017년 10월 8일 1:24

03.1.1. Secret key 발급

https://www.korbit.co.kr/settings/api 에서 Secret Key를 발급받자. Secrect Key는 발급 시 한 번만 알려주므로 잘 저장해두도록 한다. 그리고 화면에 보이는 Keys는 다음에 Client ID로 값으로 사용하게 된다.

마지막 편집일시 : 2017년 10월 16일 9:13

03.1.2. API 둘러보기

Korbit에서는 가상화폐 거래에 **Rest API** 를 제공하고 있다. **우리는 이 Rest API를 Python 이용하여 호출하게 되며 자동거래 시스템을 구축할 것이다.** REST API에 대한 상세 Spec은 아래 페이지를 참조하자.

https://apidocs.korbit.co.kr/ko/#first_section

https://apidocs.korbit.co.kr/

코빗은 산업표준으로 자리 잡은 **OAuth2.0** 을 지원하고 있다.
Rest API에 대한 문서화가 잘되어있어 문서를 참고해서 한 번씩 테스트해보자 문서에서는 curl를 이용하고 있지만, 필자는 **curl**과 Chrome의 extension에 **Advanced REST client** 두 개를 이용해서 테스트해 보도록 한다.
이후에는 Python으로 **REST API**를 Wrapping 해서 시스템화할 예정이다.

마지막 편집일시 : 2017년 10월 16일 9:13

03.1.2.1. REST API Test 도구

REST API 를 Test해볼 수 있는 tool은 굉장히 많다. Kobit의 API Reference에서는 curl을 이용한 request, response를 알려주고 있다. 이 프로젝트에서는 curl과 chrome의 확장 프로그램인 **Advanced REST client**를 이용하도록 한다.
아래 url에서 설치하면 사용할 수 있다.

https://advancedrestclient.com/

마지막 편집일시 : 2017년 10월 16일 9:14

03.1.2.2. AccessToken 발급
Refrence에 나와있는 방식대로 **linux shell** 상에서 request를 날려보면

Request: curl -d - -x post -d "client_id=$client_id&client_secret=\$client_secret&username=\$email&password=\$password&grant_type=password" https://api.korbit.co.kr/v1/oauth2/access_token

Response를 받을 수 있다.

Response: { "token_type":"Bearer", "access_token":"1t1LgTslDrGznxPxhYz7RldsNVIbnEK", "expires_in":3600, "refresh_token":"vn5xoOf4PzckgnqjQSL9Sb3KxWJvYtm" }

Response로 받은 Access token에는 API Request를 날릴 때 사용할 access token과 이 access token의 만료시간인 expires_in, refresh_token 값을 볼 수 있다. refresh_token은 access token을 3600초 이전에 갱신하기 위해 사용된다.

마지막 편집일시 : 2017년 10월 16일 9:14

03.1.2.3. Exchange: Public
API refrence상에 public에는 **Access token을 입력하지 않고도 얻을 수 있는 정보들에 대한 설명이 있다. 주로 코인의 현재 거래량, 가격정보, 그리고 코빗에서 제한하고 있는 여러 정보들에 대한 값을 얻을 수 있다. 이번에는 Advanced REST client에서 Test해보자.**

Exchange: Public에서 제공하는 항목은 현재 Ticker, Detailed Ticker, Orderbook, List of Filled Orders, Constants 5가지가 있다.

Ticker, Detailed Ticker -> 가상화폐별로 마지막 거래 정보를 알려준다.
Orderbook -> 가상화폐별 bid, ask 예약정보를 보여준다.
List of Filled Orders -> 가상화폐별로 분,시,일 단위 체결거래 정보를 보여준다.
Constants -> 가상화폐별 최소, 최대 주문 가능 정보, 출금 수수료 등 코빗에서 제한하고 있는 상수 값들에 대한 정보를 받을 수 있다.

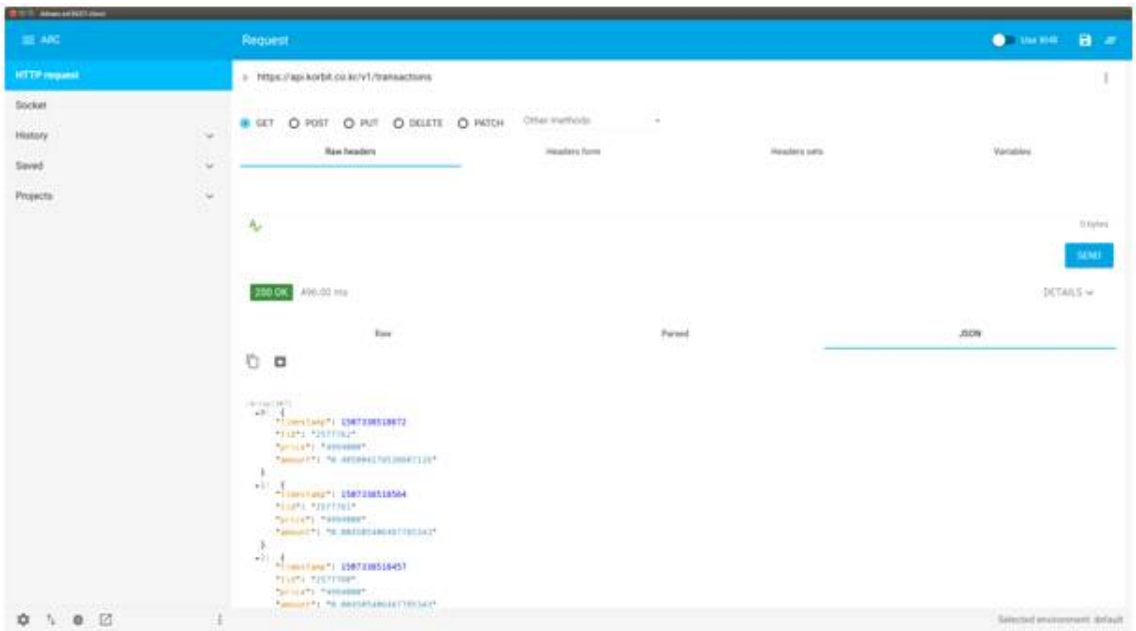
우리는 List of Filled Orders를 이용하여 가상화폐 거래정보를 DB에 저장하고 데이터를 이용하여 자동거래 시스템을 구축한다. List of Filled Orders를 한번 Test해보자.

url은 https://api.korbit.co.kr/v1/transactions 을 입력하고 method는 GET을 선택하고 Send 버튼을 눌러보자.

아래와 같이 200 OK에 response가 출력된다면 정상적으로 테스트 완료된 것이다. response에서 [ ]는 list 이며 { }는 각 항목을 의미한다.

[{"timestamp":1507338518672,"tid":"2577762","price":"4994000","amount":"0.405004278538607116"},{"timestamp":1507338518564,"tid":"2577761","price":"4994000","amount":"0.004505406487785343"},{"timestamp":1507338518457,"tid":"2577760","price":"4994000","amount":"0.004505406487785343"}, ...]

response에서 Parsed tab에 < > 를 클릭해보면 Open as JSON을 선택하여 JSON format으로 내용을 볼 수 있다.



마지막 편집일시 : 2017년 10월 16일 9:14

==========

5. 03.1.2.4. Excahnge: User

User는 사이트에서는 로그인해서 사용할 수 있는 기능들에 대한 API로 2.4.1에서 발급받은 Access token을 넘겨주고 개인 거래 정보, bid, ask 등 사고 파는 주문을 지원한다.

curl이 아닌 Chrome Advanced REST Client 프로그램을 이용해서 발급받은 Accesstoken

으로 현재 나의 bitcoin 주문 현황을 알아볼 수 있다.

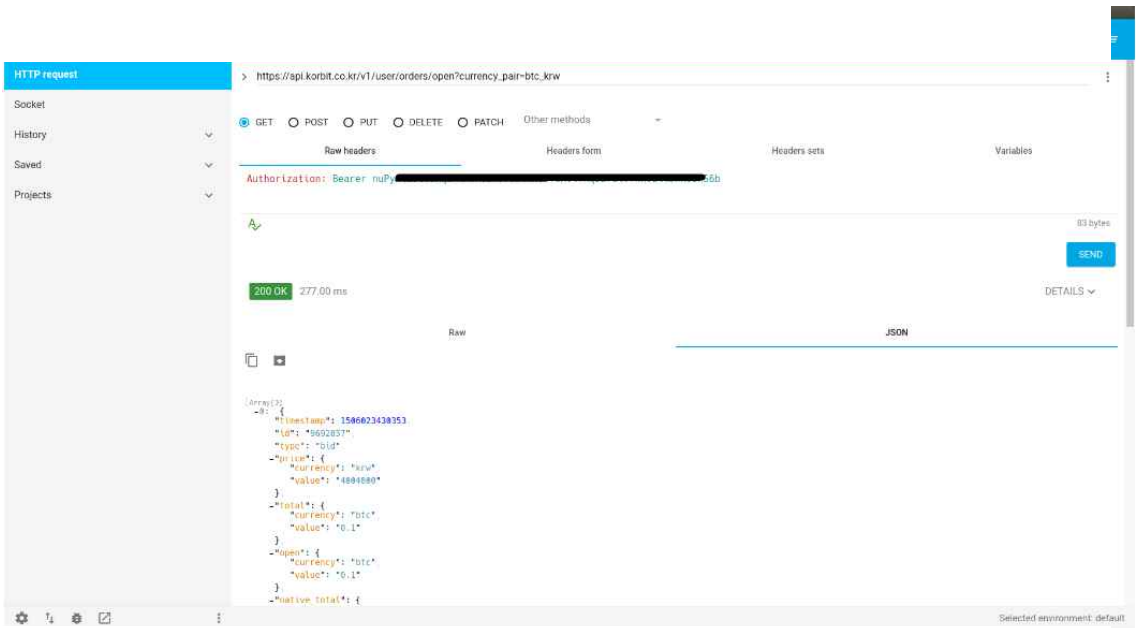
첫줄에 호출할 URL을 입력한 후 아래 GET, POST, PUT, DELETE, PATCH 중 하나를 선택한다. 우리가 호출할 Test로 호출해볼 URL은 나의 bitcoin 주문현황이다. https://api.korbit.co.kr/v1/user/orders/open?currency_pair=btc_krw

URL 입력 후 Method는 POST를 선택한다. Raw headers에 Authorization: Bearer [accessToken] 을 입력하고 Send 버튼을 누른 후 Response가 json으로 정상 출력되는지 확인한다. Raw headers를 사용하지 않고 Headers form을 사용해도 동일하다.

아래와 같이 여러 항목을 확인할 수 있다. [ ]는 list를 의미하며 comma 단위로 각 item이 구분된다.

[{"timestamp":1389173297000,"id":"58726","type":"ask","price":
{"currency":"krw","value":"800000"}, "total":
{"currency":"btc","value":"1.00000000"},"open":
{"currency":"btc","value":"0.75000000"}},
{"timestamp":1386419377000,"id":"37499","type":"bid","price":
{"currency":"krw","value":"700000"}, "total":
{"currency":"btc","value":"1.50000000"},"open":
{"currency":"btc","value":"0.41200000"}},
{"timestamp":1386419089000,"id":"37476","type":"bid","price":
{"currency":"krw","value":"10090"}, ….]

나머지 API에 대해서는 **Python으로 wrapping할때 알아보도록 하자.**



마지막 편집일시 : 2017년 10월 16일 9:14

==========

03.1.3. 거래소 모듈 구현하기

이번 장 부터는 앞서 테스트한 **REST API를 Python으로 호출**하도록 거래소 모듈을 구현한다. REST API를 call하기 위해서는 python에서는 여러가지 방법이 있다. httplib, pycurl, requests등 여러가지 library가 있다. 어떤거를 써도 무방하지만 필자는 requests를 선호하는 편이다.

시작하기 전에 requests를 설치하자. **$ pip install requests**

개발하는 모든 모듈에는 test 파일을 같이 만들어 두었다.
korbit_machine.py test_korbit_machine.py

거래소 모듈은 특별한 부분이 없다. requests 를 이용해서 korbit에서 정의해둔 parameter값만 넘겨주면 손쉽게 구현할 수 있다.

이번 장부터는 실제 셋업 과정과 구현을 진행한다. 앞서 언급했듯이 MongoDB와 관련된 내용만 해도 책 한 권 분량을 적을 수 있다. 이 프로젝트의 목표에서는 벗어나므로 여기서는 셋업 및 설정, 이 프로젝트에서 사용될 DB Operation, Query에 한정해서 다룰 예정이다. 그리고 Python으로 Wrapper를 만들어서 사용할 예정이다.

최종 구현 후 Directory 구조이다.

```
├─── README.md
├─── config
│    ├─── README.md
│    └─── config.ini
├─── machine
│    └─── korbit_machine.py
└─── tests
     └─── machine
          └─── test_korbit_machine.py
```

마지막 편집일시 : 2017년 10월 16일 9:13

03.1.3.1. Config

우선 config.ini 파일을 만들자. 소스코드에 **Key와 Password를 적어둘 수는 없으니** Config 만들어 저장해두고 불러와서 사용하도록 하자. DB가 셋업되면 DB를 이용해도 무방하다. 지금은 config.ini 파일을 만들고 아래 포맷으로 각자 개인 정보를 채우자

```
[KORBIT]
client_id =
client_secret =
username =
password =
```

마지막 편집일시 : 2017년 10월 16일 9:15

03.1.3.2. TestCase

TestCase를 작성하기 전 3.4.1.에서 만들어둔 Config 파일에서 개인정보를 읽어오도록 setUp에 작성해 둔다. 그리고 3.4.3에서 구현한 set_token을 발급받는 부분도 추가한다.

```
def setUp(self):
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        client_id = config['KORBIT']['client_id']
        client_secret = config['KORBIT']['client_secret']
        username = config['KORBIT']['username']
        password = config['KORBIT']['password']
        self.korbit_machine = KorbitMachine(mode="Prod",client_id=client_id,
                        client_secret=client_secret,
                        username=username,
                        password=password)
```

```
        token = self.korbit_machine.set_token()
```

각 장에서 구현한 부분에서 TestCase를 추가할 것이다.

마지막 편집일시 : 2017년 10월 16일 9:15

03.1.3.3. Access Token

https://apidocs.korbit.co.kr/#direct-authentication

Korbit에서 제공하는 Exchange: User 부분을 이용하기 위해서는 Access Token을 생성해야 한다. Reference의 내용을 그대로 코드로 구현하였다.

set_token 메소드를 만들고 grant type별로 REST API 호출 시 넘겨줄 data를 저장한 후 API를 call한다.

```
def set_token(self, grant_type="password"):
        """
        set token
        grant_type: password, refresh_token

        saved self.access_token, self.refresh_token, self.token_type
        """
        token_api_path = "/v1/oauth2/access_token"

        url_path = self.host + token_api_path
        if grant_type == "password":
            data = {"client_id":self.client_id,
                    "client_secret":self.client_secret,
                    "username":self.username,
                    "password":self.password,
                    "grant_type":grant_type}
        elif grant_type == "refresh_token":
            data = {"client_id":self.client_id,
                    "client_secret":self.client_secret,
                    "refresh_token":self.refresh_token,
                    "grant_type":grant_type}
        else:
             raise Exception("Unexpected grant_type")
        res = requests.post(url_path, data=data)
        result = res.json()
        self.access_token = result["access_token"]
        self.token_type = result["token_type"]
        self.refresh_token = result["refresh_token"]
        return self.access_token
```

API를 call 한 후 Response는 json으로 넘겨 받고 결과값을 저장한다. 결과값은 Class의 멤버 변수로 저장해두고 사용한다.

Test Case를 작성해보자

```
    def test_set_token(self):
        token = self.kobit_machine.set_token()
        assert token
        print(token)
        refresh_token = self.korbit_machine.set_token(grant_type="refresh_token")
        print(refresh_token)
        assert refresh_token
```

마지막 편집일시 : 2017년 10월 16일 9:15

03.1.3.4. Ticker
https://apidocs.korbit.co.kr/#ticker https://apidocs.korbit.co.kr/#detailed-ticker

tick을 얻어오는 부분으로 api상에서 2가지 부분을 request_type으로 받아서 url을 조립하는 방식으로 구현하였다. 넘겨줄 parameter는 조회하고 싶은 코인의 종류를 넘겨주면 된다.

```
    def get_ticker(self, coin_type=None):
        """
        Get Ticker

        args:
            coin_type

        Returns:
            timestamp
            last
            high
            low
            volume
        """
        if coin_type is None:
            raise Exception('Need to coin type')
        time.sleep(1)
        payload = {'currency_pair':coin_type}
        ticker_api_path = "/v1/ticker/detailed"
        url_path = self.host + ticker_api_path
        res = requests.get(url_path, params=payload)
        response_json = res.json()
        result={}
        result["timestamp"] = str(response_json["timestamp"])
        result["last"] = response_json["last"]
        result["high"] = response_json["high"]
        result["low"] = response_json["low"]
        result["volume"] = response_json["volume"]
        return result
```

TestCase 를 추가해보자.

```
    def test_get_ticker(self):
        """
        test get_ticker
        request_type = None, detailed
        """
        print(inspect.stack()[0][3])
        ticker = self.kobit_machine.get_ticker("etc_krw")
        assert ticker
        print(ticker)
        detailed_ticker = self.korbit_machine.get_ticker("etc_krw", "detailed")
        assert detailed_ticker
        print(detailed_ticker)
```

Result {'timestamp': 1507430886491, 'last': '14010'} {'ask': '14000', 'low': '13830', 'high': '14050', 'bid': '13980', 'last': '14010', 'timestamp': 1507430886491, 'change': '10', 'changePercent': '0.07', 'volume': '67983.40048533429142621'}

마지막 편집일시 : 2017년 11월 4일 6:31

03.1.3.5. Filled Orders
https://apidocs.korbit.co.kr/#list-of-filled-orders

최근 1분동안 코인 거래 정보를 조회할 수 있다. 이 부분을 추후에 DB에 저장할 것이다. 조회가능한 범위는 최근 1분, 1시간, 1일 데이터이다. parameter로 받아서 처리하도록 한다.

```
    def get_filled_orders(self, coin_type=None, per="minute"):
        """
        List of filled orders
```

```
        Keyword arguments:
        per -- last time minute,hour,day

        Returns:
            api response
        """
        if coin_type is None:
            raise Exception("Need to coin_type")
        time.sleep(1)
        payload = {'currency_pair':coin_type,'time':per}
        orders_api_path = "/v1/transactions"
        url_path = self.host + orders_api_path
        res = requests.get(url_path, params=payload)
        result = res.json()
        return result
```

TestCase

```
        def test_get_filled_orders(self):
        """
        test get_filled_orders
        """
        print(inspect.stack()[0][3])
        order_book = self.korbit_machine.get_filled_orders(coin_type="btc_krw")
        print(order_book)
        assert order_book
```

Result [{'price': '5027000', 'amount': '0.257728267423910885', 'tid': '2583577', 'timestamp': 1507432164919}, {'price': '5027000', 'amount': '0.004475830515217823', 'tid': '2583576', 'timestamp':1507432164810}, {'price': '5027000', 'amount': '0.004475830515217823', 'tid': '2583575', 'timestamp': 1507432164702}, {'price': '5027000', 'amount': '0.004475830515217823', 'tid': '2583574', 'timestamp': 1507432164594}, {'price': '5027000', 'amount': '0.01989258', 'tid': '2583573', 'timestamp': 1507432164445}, {'price': '5027000', 'amount': '0.004475830515217823', 'tid': '2583572', 'timestamp': 1507432164335}, {'price': '5027000', 'amount': '0.004475830515217823', 'tid': '2583571', 'timestamp': 1507432164228}, {'price': '5027500', 'amount': '0.02', 'tid': '2583570', 'timestamp': 1507432122750}] 리스트 형식으로 값을 돌려준다.

마지막 편집일시 : 2017년 11월 4일 6:34

03.1.3.6. Constants
https://apidocs.korbit.co.kr/#constants

거래에 필요한 정보를 받아온다. 호가 단위 및 최소, 최대 주문량 등을 조회할 수 있다.

```
    def get_constants(self):
        """
        Get constant values

        Keyword arguments:

        Returns:
            api response
        """
        time.sleep(1)
        constants_api_path = "/v1/constants"
        url_path = self.host + constants_api_path
        res = requests.get(url_path)
        result = res.json()
        self.constants = result
        return result
```

TestCase

```
    def test_get_constants(self):
        """
        test get_constants
        saved self.constants
        """
        print(inspect.stack()[0][3])
        constants = self.korbit_machine.get_constants()
        assert constants
        print(constants)
```

Result: {'maxBtcWithdrawal': 5, 'minBtcOrder': 0.001, 'minEthOrder': 0.01, 'minTradableLevel': 2, 'minEthPrice': 1000, 'maxEtcOrder': 20000, 'minEtcOrder': 0.1, 'ethTickSize': 50, 'maxKrwWithdrawal': 5000000000, 'maxBtcOrder': 100, 'maxEthOrder': 1000, 'maxEtcPrice': 100000000, 'etcTickSize': 10, 'minEtcPrice': 100, 'minBtcPrice': 1000, 'btcTickSize': 500, 'maxEthPrice': 100000000, 'btcWithdrawalFee': 0.001, 'minKrwWithdrawal': 1000, 'minBtcWithdrawal': 0.0001, 'krwWithdrawalFee': 1000, 'maxBtcPrice': 100000000}

위와 같이 나오면 정상이다.

마지막 편집일시 : 2017년 10월 16일 9:15

03.1.3.7. Wallet Status
https://apidocs.korbit.co.kr/#check-user's-balances-(upcoming-feature-for-8/16/2017-update)

이 response를 보면 각 코인에 대한 보유 현황 및 사용가능한 원화도 확인할 수 있다.

```
  def get_wallet_status(self):
        """
        Get wallet_status
        """
        time.sleep(1)
        wallet_status_api_path ="/v1/user/balances"
        url_path = self.host + wallet_status_api_path
        headers = {"Authorization":"Bearer " + self.access_token}
        res = requests.get(url_path, headers=headers)
        result = res.json()
        print("balance:"+result["krw"]["available"])
        return result["krw"]["available"]
```

TestCase

```
    def test_get_wallet_status(self):
        """
        test get_wallet_status
        """
        print(inspect.stack()[0][3])
        wallet_status = self.korbit_machine.get_wallet_status()
        assert wallet_status
        print(wallet_status)
```

Result {'xmr': {'available': '0', 'trade_in_use': '0', 'withdrawal_in_use': '0'}, 'bch': {'available': '0', 'trade_in_use': 'x.xxx', 'withdrawal_in_use': '0'}, 'rep': {'available': '0', 'trade_in_use': '0', 'withdrawal_in_use': '0'}, 'dash': {'available': '0', 'trade_in_use': '0', 'withdrawal_in_use': '0'}, 'eth': {'available': 'x.xxx', 'trade_in_use': 'x.xxxx', 'withdrawal_in_use': '0'}, 'steem': {'available': '0', 'trade_in_use': '0', 'withdrawal_in_use': '0'}, 'xrp': {'available': '0', 'trade_in_use': 'xxxxx.xxxx', 'withdrawal_in_use': '0'}, 'zec': {'available': '0', 'trade_in_use': '0', 'withdrawal_in_use': '0'}, 'etc': {'available': 'xx.xxx', 'trade_in_use': 'x.xxxx', 'withdrawal_in_use': '0'}, 'krw': {'available': 'xxxxxxxx', 'trade_in_use': 'xxxxxxx', 'withdrawal_in_use': '0'}, 'ltc': {'available': '0', 'trade_in_use': '0', 'withdrawal_in_use': '0'}, 'btc': {'available': 'x.xxxxxxx', 'trade_in_use': 'x.xxxxxxx',

'withdrawal_in_use': '0'}} balance:xxxxxx

마지막 편집일시 : 2017년 10월 16일 9:15

03.1.3.8. User Orders
https://apidocs.korbit.co.kr/#list-open-orders

현재 나의 주문현황을 조회할 수 있다. 여기서 나의 전체 주문현황과 하나하나 항목의 상세 정보를 볼 수 있다.

```
    def get_list_my_orders(self, coin_type=None):
        """
        Get my order list
        """
        if coin_type is None:
            raise Exception("Need to coin_type")
        time.sleep(1)
        payload = {'currency_pair':coin_type}
        list_order_api_path = "/v1/user/orders/open"
        url_path = self.host + list_order_api_path
        headers = {"Authorization":"Bearer " + self.access_token}
        res = requests.get(url_path, headers=headers, params=payload)
        result = res.json()
        return result
```

TestCase

```
    def test_get_list_my_orders(self):
        """
        test get_list_my_order
        """
        print(inspect.stack()[0][3])
        my_orders = self.korbit_machine.get_list_my_orders(coin_type="btc_krw")
        assert my_orders
        print(my_orders)
```

Return [{'type': 'ask', 'price': {'value': '5100000', 'currency': 'krw'}, 'id': '10203121', 'total': {'value': '0.09992', 'currency': 'btc'}, 'open': {'value': '0.09992', 'currency': 'btc'}, 'native_total': {'value': '509592', 'currency': 'krw'}, 'timestamp': 1507177528636}, {'type': 'ask', 'price': {'value': '5400000', 'currency': 'krw'}, 'id': '10039302', 'total': {'value': '0.0998', 'currency': 'btc'}, 'open': {'value': '0.0998', 'currency': 'btc'}, 'native_total': {'value': '538920', 'currency': 'krw'}, 'timestamp': 1506812423195}, {'type': 'ask', 'price': {'value': '5300000', 'currency': 'krw'}, 'id': '10015147', 'total': {'value': '0.0999', 'currency': 'btc'}, 'open': {'value': '0.0999', 'currency': 'btc'}, 'native_total': {'value': '529470', 'currency': 'krw'}, 'timestamp': 1506758424835}, {'type': 'ask', 'price': {'value': '5200000', 'currency': 'krw'}, 'id': '9916644', 'total': {'value': '0.0999', 'currency': 'btc'}, 'open': {'value': '0.0999', 'currency': 'btc'}, 'native_total': {'value': '519480', 'currency': 'krw'}, 'timestamp': 1506569420641}]

여기서 보여지는 id는 이용하여 주문을 Cancel 할 수 있다.

```
def get_my_order_status(self, coin_type=None, order_id=None):
        """
        get list my transaction history
        """
        if coin_type is None or order_id is None:
            raise Exception("Need to currency_pair and order id")
        time.sleep(1)
        list_transaction_api_path = "/v1/user/orders?currency_pair=" + coin_type
        list_transaction_api_path += "&id=" + order_id
        url_path = self.host + list_transaction_api_path
        headers = {"Authorization":"Bearer " + self.access_token}
        res = requests.get(url_path, headers=headers)
        result = res.json()
```

```
        return result
```

Test

```
    def test_get_my_order_status(self):
        """
        test get_list_my_order
        """
        print(inspect.stack()[0][3])
        my_order = self.korbit_machine.get_my_order_status("etc_krw",'5041345')
        assert my_order
        print(my_order)
```

Result

[{'id': '5041345', 'currency_pair': 'etc_krw', 'side': 'bid', 'avg_price': '12800', 'price': '12800', 'order_amount': '5', 'filled_amount': '5', 'order_total': '64000', 'filled_total': '64000', 'created_at': 1507166111816, 'last_filled_at': 1508312196789, 'status': 'filled', 'fee': '0.004'}]

여기서 주의 깊게 봐야할 부분은 하나의 주문에 조회를 할때에도 list로 결과 값이 return 된다. 그 이유는 1개의 코인도 분할매수, 분할매도가 가능하기 때문에 분할거래가 된 상태에서는 분할되어진 코인의 각 status가 partial_filled, unfilled가 된다. 그렇기 때문에 list로 return값이 오게 된다.

마지막 편집일시 : 2017년 11월 4일 6:53

03.1.3.9. Order
여기서부터는 실제 bid, ask, cancel 주문 모듈을 만든다. 코빗은 API로 거래를 지원하는 건 btc, etc, eth 3종류이다. xrp는 가격 정보, 체결현황등까지는 조회가 가능하지만 거래는 아직 지원하지 않고 있다.

Order를 내는 API에는 다른 API와는 다른게 nonce 부분이 추가되어야 한다. nonce 관한 내용은 링크를 따라 확인해보자 nonce는 증가하는 값을 넘겨줘야한다. 시간을 string으로 변환하여 넘겨주도록 하자.

```
    def get_nonce(self):
        return str(int(time.time()))
```

마지막 편집일시 : 2017년 11월 4일 6:37

03.1.3.9.1. Buy
https://apidocs.korbit.co.kr/#place-a-bid-order

코인을 사자 주문을 구현해보자. 코인 종류, 살 가격, 수량, 주문 타입을 parameter로 넘긴다. 현재 코빗에서 api로 지원하는 건 btc, etc, eth 3종류가 있다. 그리고 주문 타입은 limit은 지정가 거래이고 market은 시장가 거래로 즉시 거래되며 flat_amount를 추가로 넘겨줘야 한다. 이번 프로젝트에서는 지정가 거래만 구현한다. 시장가 거래도 구현하면 되지만 굳이 쓸 계획이 없어 구현하지는 않았다. 시장가 거래가 필요하다면 직접 구현해보도록 하자.

```
        def buy_coin_order(self, coin_type=None, price=None, qty=None, order_type="limit"):
        """
        buy_coin_order
        """
        time.sleep(1)
        if coin_type is None or price is None or qty is None:
            raise Exception("Need to param")
        buy_coin_order_api_path = "/v1/user/orders/buy"
        url_path = self.host + buy_coin_order_api_path
        headers = {"Authorization":"Bearer " + self.access_token}
        data = {"currency_pair":coin_type,
                "type":order_type,
                "price":price,
                "coin_amount":qty,
```

```
                "nonce":self.get_nonce()}
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        return result
```

TestCase

```
 def test_buy_coin_order(self):
        print(inspect.stack()[0][3])
        buy_order = self.korbit_machine.buy_coin_order(coin_type="etc_krw", price="12000", qty="10", order_type="limit")
        assert buy_order
        print(buy_order)
```

Result {'currencyPair': 'etc_krw', 'status': 'success', 'orderId': 5064611}

12000원에 etc 코인을 10개 주문했다. orderId는 이후에 Cancel할때 사용할 수 있다. 그리고 주문이 정상적으로 되었는지는 Web상에서 확인하거나 3.4.8에서 구현한 User orders에서 확인할 수 있다. 추가로 status에 success가 아니면 요청이 실패된 것이다. 주문 실패는 여러가지 요인으로 한 번에 처리안될 수 있다. 주문이 폭주, 네트워크 상태, 서버상태등 여러가지 원인으로 실패할 수 있으니 status를 꼭 확인할 수 있도록 하자.

마지막 편집일시 : 2017년 11월 4일 6:43

03.1.3.9.2. Sell
코인을 팔기 위한 메소드를 추가해보자. 3.4.7에서 구현한 Wallet status에서 코인 종류별로 available에 해당하는 수량만큼 원하는 가격에 주문을 낼 수 있다.

```
    def sell_coin_order(self, coin_type=None, price=None, qty=None, order_type="limit"):
        """
        buy_coin_order
        """
        time.sleep(1)
        if price is None or qty is None or coin_type is None:
            raise Exception("Need to params")
        if order_type != "limit":
            raise Exception("Check order type")
        buy_coin_order_api_path = "/v1/user/orders/sell"
        url_path = self.host + buy_coin_order_api_path
        headers = {"Authorization":"Bearer " + self.access_token}
        data = {"currency_pair":coin_type,
                "type":order_type,
                "price":price,
                "coin_amount":qty,
                "nonce":self.get_nonce()}
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        return result
```

TestCase

```
    def test_sell_coin_order(self):
        print(inspect.stack()[0][3])
        sell_order = self.korbit_machine.sell_coin_order(coin_type="etc_krw", price="20000", qty="10", order_type="limit")
        assert sell_order
        print(sell_order)
```

Result {'currencyPair': 'etc_krw', 'status': 'success', 'orderId': 5064650}

Buy, Sell, Cancel 모두 response에서 status를 체크해야 한다.

마지막 편집일시 : 2017년 11월 4일 6:43

03.1.3.9.3. Cancel
주문을 취소하는 메소드를 구현해보자. Buy, Sell 주문 시 받은 Response에 orderId를

parameter로 넘겨주자. Cancel은 다른 주문과는 다르게 여러 OrderId를 넘겨서 취소 처리할 수 있다. 여기서는 하나씩만 요청하는 걸로 구현했다.

```
def cancel_coin_order(self, coin_type=None, price=None, qty=None, order_type=None, order_id=None):
        """
        cancel_coin_order
        """
        time.sleep(1)
        if coin_type is None or order_id is None:
            raise Exception("Need to params")
        cancel_coin_order_api_path = "/v1/user/orders/cancel"
        url_path = self.host + cancel_coin_order_api_path
        headers = {"Authorization":"Bearer " + self.access_token}
        data = {"currency_pair":coin_type,
                "id":order_id,
                "nonce":self.get_nonce()}
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        return result
TestCase
        def test_cacnel_coin_order(self):
        print(inspect.stack()[0][3])
        cancel_order = self.korbit_machine.cancel_coin_order(coin_type="etc_krw", order_id="5064611")
        assert cancel_order
        print(cancel_order)
Result test_kobit_machine [{'currencyPair': 'etc_krw', 'status': 'success', 'orderId': '5064611'}]
```

response는 list로 돌려주며 status에 주문처리 결과를 알려준다.

마지막 편집일시 : 2017년 11월 4일 6:43
==========

03.2. CoinOne

CoinOne의 V1은 BTC만 지원하는 관계로 여러 코인을 지원하는 API V2를 기준으로 설명한다. Korbit과 비슷하지만 미묘하게 다른 부분들이 있다. coinone의 API guide에서는 httplib2를 이용해 REST API를 call하지만 필자는 Korbit에서 사용한 request를 그대로 사용하겠다.

마지막 편집일시 : 2017년 10월 18일 10:0

6. 03.2.1. API 발급

통합 API 관리에서 V2로 API를 만들자.

거래소 조회, 사용자 조회, 거래소 주문 3가지 체크하자.

생성 후 API Key는 Email로 받을 수 있다. Secret_key와 Access_token은 cofig.ini파일에 저장하자.

아래 포맷으로 저장한다.

[COINONE_V2]

access_token =

secret_key =

마지막 편집일시 : 2017년 10월 18일 9:56

03.2.2. 거래소 모듈 구현하기

코인원 거래소를 이용하기 위한 거래모듈을 구현한다. Korbit과 함께 쓸 수 abstract class를 사용하고, result값에 대한 공통 설계가 필요하다.

마지막 편집일시 : 2017년 10월 26일 1:14

03.2.2.1. Access Token

Korbit과 다른 점은 Korbit은 client_id와 secret_key로 access token를 받아서 사용하지만 CoinOne은 초기에 개인용 API를 생성할때 access token을 email로 보내 준다. 이 토큰을 이용해서 api를 call하게 되지만 30일이 만료고 그 전에 token을 refresh해야 한다. 그래서 필자는 거래 시 마다 refresh token으로 발급받고 그 access token을 config에 저장하고 다음 호출에서 다시 refresh하고 새로 발급 받은 access token은 config에 저장하는 방식으로 구현하였다. parameter가 refresh_token이 아니면 현재 config에 저장된 access token을 return 한다.

```
token_api_path =

url_path = self.host + token_api_path


url_path = self.host + token_api_path
if grant_type ==

url_path = self.host + token_api_path
if grant_type == "refresh_token":
    headers = {

url_path = self.host + token_api_path
if grant_type == "refresh_token":
    headers = {"content-type":

url_path = self.host + token_api_path
if grant_type == "refresh_token":
    headers = {"content-type":"application/x-www-form-urlencoded"}
    data = {
```

```
url_path = self.host + token_api_path
if grant_type == "refresh_token":
    headers = {"content-type":"application/x-www-form-urlencoded"}
    data = {"access_token":self.access_token}
    config = configparser.ConfigParser()
    config.read(

url_path = self.host + token_api_path
if grant_type == "refresh_token":
    headers = {"content-type":"application/x-www-form-urlencoded"}
    data = {"access_token":self.access_token}
    config = configparser.ConfigParser()
    config.read('conf/config.ini')
    res = requests.post(url_path, headers=headers, data=data)
    result = res.json()
    self.access_token = result[

url_path = self.host + token_api_path
if grant_type == "refresh_token":
    headers = {"content-type":"application/x-www-form-urlencoded"}
    data = {"access_token":self.access_token}
    config = configparser.ConfigParser()
    config.read('conf/config.ini')
    res = requests.post(url_path, headers=headers, data=data)
    result = res.json()
    self.access_token = result["accessToken"]
    config[

url_path = self.host + token_api_path
if grant_type == "refresh_token":
    headers = {"content-type":"application/x-www-form-urlencoded"}
    data = {"access_token":self.access_token}
    config = configparser.ConfigParser()
    config.read('conf/config.ini')
    res = requests.post(url_path, headers=headers, data=data)
    result = res.json()
    self.access_token = result["accessToken"]
    config["COINONE"][

url_path = self.host + token_api_path
if grant_type == "refresh_token":
```

```
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token


    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open(

    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini',

    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
```

```
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w')

    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w') as configfile:
            config.write(configfile)


    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w') as configfile:
            config.write(configfile)
    else:
        config = configparser.ConfigParser()
        config.read(


    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
```

```
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w') as configfile:
            config.write(configfile)
    else:
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        self.access_token = config[

    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w') as configfile:
            config.write(configfile)
    else:
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        self.access_token = config['COINONE'][

    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w') as configfile:
            config.write(configfile)
    else:
```

```
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        self.access_token = config['COINONE']['access_token']
        print(

    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w') as configfile:
            config.write(configfile)
    else:
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        self.access_token = config['COINONE']['access_token']
        print("get confing.ini")
        print(self.access_token)


    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w') as configfile:
            config.write(configfile)
    else:
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        self.access_token = config['COINONE']['access_token']
```

```
        print("get confing.ini")
        print(self.access_token)
    return self.access_token



    url_path = self.host + token_api_path
    if grant_type == "refresh_token":
        headers = {"content-type":"application/x-www-form-urlencoded"}
        data = {"access_token":self.access_token}
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        res = requests.post(url_path, headers=headers, data=data)
        result = res.json()
        self.access_token = result["accessToken"]
        config["COINONE"]["access_token"] = self.access_token
        with open('conf/config.ini', 'w') as configfile:
            config.write(configfile)
    else:
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        self.access_token = config['COINONE']['access_token']
        print("get confing.ini")
        print(self.access_token)
    return self.access_token
```

Testcase

```
    print(inspect.stack()[
    print(inspect.stack()[0][
    print(inspect.stack()[0][3])
    access_token = self.coinone_machine.set_token(grant_type=
    print(inspect.stack()[0][3])
    access_token = self.coinone_machine.set_token(grant_type="refresh_token")

    print(inspect.stack()[0][3])
    access_token = self.coinone_machine.set_token(grant_type="refresh_token")
    assert access_token
```

마지막 편집일시 : 2017년 10월 26일 1:14

03.2.2.2. Wallet Status

http://doc.coinone.co.kr/#api-Account_V2-UserBalance

현재 나의 Coin 및 원화 잔고를 확인할 수 있는 API이다. 기본적으로 reference에 가이드하는 방법을 따르지만 조금 차이가 있다. 필자는 python3를 기본으로 사용하였기 때문에. Unicode가 기본이다. 그래서 encode('utf-8')로 str객체로 변환 시켜서 signature를 encoded_payload를 생성해야 한다.

```
signature = hmac.new(secret_key, encoded_payload, hashlib.sha512);

signature = hmac.new(secret_key, encoded_payload, hashlib.sha512);
return signature.hexdigest()



time.sleep(
time.sleep(1)
wallet_status_api_path =
url_path = self.host + wallet_status_api_path
payload ={

url_path = self.host + wallet_status_api_path
payload ={
    "access_token":self.access_token,

url_path = self.host + wallet_status_api_path
payload ={
    "access_token":self.access_token,
    'nonce':self.get_nonce()
}
dumped_json = json.dumps(payload)
encoded_payload = base64.b64encode(dumped_json.encode(
url_path = self.host + wallet_status_api_path
payload ={
    "access_token":self.access_token,
    'nonce':self.get_nonce()
}
dumped_json = json.dumps(payload)
encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
encoded_secret_key = base64.b64encode(self.secret_key.upper().encode(
```

```
        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
        encoded_secret_key =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {
        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
        encoded_secret_key =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {'Content-type':
        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
        encoded_secret_key =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {'Content-type': 'application/json',

        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
```

```
        encoded_secret_key                                                    =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,

        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
        encoded_secret_key                                                    =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE':      self.get_signature(encoded_payload,
self.secret_key.encode(
        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
        encoded_secret_key                                                    =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE':      self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()

        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
```

```
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
        encoded_secret_key =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE': self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()
        return result


        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
        encoded_secret_key =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE': self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()
        return result
```

Result

{'btc': {'avail': '0.00000000', 'balance': '0.00000000'}, 'krw': {'avail': '0', 'balance': '0'}, 'errorCode': '0', 'bch': {'avail': '0.00000000', 'balance': '0.00000000'}, 'normalWallets': [], 'xrp': {'avail': '0.00000000', 'balance': '0.00000000'}, 'result': 'success', 'eth': {'avail': '0.00000000', 'balance': '0.00000000'}, 'qtum': {'avail': '0.00000000', 'balance': '0.00000000'}, 'etc': {'avail': '0.00000000', 'balance':

'0.00000000'}}

```
        url_path = self.host + wallet_status_api_path
        payload ={
            "access_token":self.access_token,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))
        encoded_secret_key =
base64.b64encode(self.secret_key.upper().encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE': self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()
        return result
```

Result

{'btc': {'avail': '0.00000000', 'balance': '0.00000000'}, 'krw': {'avail': '0', 'balance': '0'}, 'errorCode': '0', 'bch': {'avail': '0.00000000', 'balance': '0.00000000'}, 'normalWallets': [], 'xrp': {'avail': '0.00000000', 'balance': '0.00000000'}, 'result': 'success', 'eth': {'avail': '0.00000000', 'balance': '0.00000000'}, 'qtum': {'avail': '0.00000000', 'balance': '0.00000000'}, 'etc': {'avail': '0.00000000', 'balance': '0.00000000'}}

TestCase

```
        print(inspect.stack()[
        print(inspect.stack()[0][
        print(inspect.stack()[0][3])
        result = self.coinone_machine.get_wallet_status()

        print(inspect.stack()[0][3])
        result = self.coinone_machine.get_wallet_status()
        assert result
        print(result)
```

마지막 편집일시 : 2017년 10월 26일 1:14

03.2.2.3. Buy Order
http://doc.coinone.co.kr/#api-Order_V2-LimitBuy

코인별로 Buy 주문을 내는 API 이다. Token을 구현했다면 이후부터는 대동소이하기 때문에 구현에 큰 어려움은 없다. 다만 추후에 Korbit 모듈과 호환을 이루기 위해서는 return 값을 맞출 필요가 있다. 이부분은 추후에 다시 진행하겠다.

```
    def buy_coin_order(self, coin_type="btc", price=None, coin_amount=None, order_type="limit"):
        """
        buy_coin_order
        """
        if order_type != "limit":
            raise Exception("Coinone order type support only limit.")
        time.sleep(1)
        buy_limit_api_path ="/v2/order/limit_buy/"
        url_path = self.host + buy_limit_api_path
        payload ={
            "access_token" : self.access_token,
            "price" : int(price),
            "qty" : float(coin_amount),
            "currency":coin_type,
            'nonce' : self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE': self.get_signature(encoded_payload, self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()
        return result
Result {'errorCode': '0', 'orderId': '60066de5-bd9f-4fcf-8381-f623d4exxxxx', 'result': 'success'}
```

TestCase

def test_buy_coin_order(self): print(inspect.stack()[0][3]) result = self.coinone_machine.buy_coin_order(coin_type="xrp", price="230", coin_amount="1", order_type="limit") assert result print(result)

```

마지막 편집일시 : 2017년 10월 26일 1:14
---

# 03.2.2.4. Sell Order

http://doc.coinone.co.kr/#api-Order_V2-LimitSell
코인별로 Sell 주문을 내는 API 이다. Buy 부분과는 api path만 다르고 다 동일하다.

```
if order_type !=
if order_type != "limit":

if order_type != "limit":
    raise Exception(
if order_type != "limit":
    raise Exception("Coinone order type support only limit.")
time.sleep(
if order_type != "limit":
    raise Exception("Coinone order type support only limit.")
time.sleep(1)
sell_limit_api_path =
url_path = self.host + sell_limit_api_path
payload ={

url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,

url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,
    "price" : int(price),
```
```

```
url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,
    "price" : int(price),
    "qty" : float(coin_amount),

url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,
    "price" : int(price),
    "qty" : float(coin_amount),
    "currency":coin_type,

url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,
    "price" : int(price),
    "qty" : float(coin_amount),
    "currency":coin_type,
    'nonce' : self.get_nonce()
}

dumped_json = json.dumps(payload)
encoded_payload = base64.b64encode(dumped_json.encode(
url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,
    "price" : int(price),
    "qty" : float(coin_amount),
    "currency":coin_type,
    'nonce' : self.get_nonce()
}

dumped_json = json.dumps(payload)
encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))


headers = {
url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,
```

```
    "price" : int(price),
    "qty" : float(coin_amount),
    "currency":coin_type,
    'nonce' : self.get_nonce()
}

dumped_json = json.dumps(payload)
encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))


headers = {'Content-type':
url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,
    "price" : int(price),
    "qty" : float(coin_amount),
    "currency":coin_type,
    'nonce' : self.get_nonce()
}

dumped_json = json.dumps(payload)
encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))


headers = {'Content-type': 'application/json',

url_path = self.host + sell_limit_api_path
payload ={
    "access_token" : self.access_token,
    "price" : int(price),
    "qty" : float(coin_amount),
    "currency":coin_type,
    'nonce' : self.get_nonce()
}

dumped_json = json.dumps(payload)
encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))


headers = {'Content-type': 'application/json',
           'X-COINONE-PAYLOAD': encoded_payload,
```

```
        url_path = self.host + sell_limit_api_path
        payload ={
            "access_token" : self.access_token,
            "price" : int(price),
            "qty" : float(coin_amount),
            "currency":coin_type,
            'nonce' : self.get_nonce()
        }

        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))


        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE':      self.get_signature(encoded_payload,
self.secret_key.encode(
        url_path = self.host + sell_limit_api_path
        payload ={
            "access_token" : self.access_token,
            "price" : int(price),
            "qty" : float(coin_amount),
            "currency":coin_type,
            'nonce' : self.get_nonce()
        }

        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))


        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE':      self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()

        url_path = self.host + sell_limit_api_path
        payload ={
```

```
            "access_token" : self.access_token,
            "price" : int(price),
            "qty" : float(coin_amount),
            "currency":coin_type,
            'nonce' : self.get_nonce()
        }

        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))


        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE':      self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()
        return result
```

Result {'errorCode': '0', 'orderId': '60066de5-bd9f-4fcf-8381-f623d4exxxxx', 'result': 'success'}

TestCase

```
def test_sell_coin_order(self): print(inspect.stack()[0][3]) result = self.coinone_machine.sell_coin_order(coin_type="xrp", price="230", coin_amount="1", order_type="limit") assert result print(result)
```

마지막 편집일시 : 2017년 10월 26일 1:14

03.2.2.5. Cancel Order

http://doc.coinone.co.kr/#api-Order_V2-CancelOrder

Buy, Sell과 크게 다르지 않다. 다만 sell 주문을 취소하려면 is_ask에 1을 넣어줘야되고 order_id를 추가로 던저줘야 한다.

```
    def cancel_coin_order(self, coin_type=None, price=None, qty=None,
order_type=None, order_id=None):
        """
        cancel_coin_order
        """
        if coin_type is None or price is None or qty is None or order_type is None
```

```
or order_id is None:
            raise Exception("Need to parameter")
        time.sleep(1)
        cancel_api_path ="/v2/order/cancel/"
        url_path = self.host + cancel_api_path
        if order_type == "sell":
            is_ask = 1
        else:
            is_ask = 0
        payload ={
            "access_token" : self.access_token,
            "order_id" : order_id,
            "price" : int(price),
            "qty" : float(qty),
            "currency":coin_type,
            "is_ask": is_ask,
            'nonce' : self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE': self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()
        return result
```

Result

```
{'errorCode': '0', 'result': 'success'}
```

TestCase

```
def test_cancel_coin_order(self):
        print(inspect.stack()[0][3])
        result = self.coinone_machine.cancel_coin_order(coin_type="xrp",
                                                        price="230",
                                                        qty="1",
                                                        order_type="buy",
```

```
order_id="76289afa-2635-4c6e-8ce4-38d67xxxxxxx")
        assert result
        print(result)
```

마지막 편집일시 : 2017년 10월 26일 11:35

03.2.2.5. User Orders

http://doc.coinone.co.kr/#api-Order_V2-LimitOrders

나의 코인별 주문현황을 조회할 수 있다.

```
      def get_list_my_orders(self, coin_type=None):
        """
        Get my order list
        """
        list_api_path = "/v2/order/limit_orders/"
        url_path = self.host + list_api_path
        payload ={
            "access_token":self.access_token,
            "currency":coin_type,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE': self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()
        return result
```

Result

{'errorCode': '0', 'result': 'success', 'limitOrders': [{'orderId': '60066de5-bd9f-4fcf-8381-f623d4xxxxxx', 'index': '0', 'timestamp': '1508945672', 'price': '230', 'qty': '1.0000', 'feeRate': '0.001', 'type': 'bid'}]}

TestCase

```
    def test_get_list_my_orders(self):
        print(inspect.stack()[0][3])
```

```
        result = self.coinone_machine.get_list_my_orders(coin_type="xrp")
        assert result
        print(result)
```

마지막 편집일시 : 2017년 10월 28일 9:22

03.2.2.6. Order Status

http://doc.coinone.co.kr/#api-Order_V2-MyOrderInfo

유저 개별 주문의 상태를 확인할 수 있다.

Reference를 참조하면 status값에 따라 거래 체결 유무등을 확인할 수 있다.

```
    def get_my_order_status(self, coin_type=None, order_id=None):
        """
        get list my transaction history
        """
        list_api_path = "/v2/order/order_info/"
        url_path = self.host + list_api_path
        payload ={
            "access_token":self.access_token,
            "currency":coin_type,
            "order_id":order_id,
            'nonce':self.get_nonce()
        }
        dumped_json = json.dumps(payload)
        encoded_payload = base64.b64encode(dumped_json.encode('utf-8'))

        headers = {'Content-type': 'application/json',
                   'X-COINONE-PAYLOAD': encoded_payload,
                   'X-COINONE-SIGNATURE': self.get_signature(encoded_payload,
self.secret_key.encode('utf-8'))}

        res = requests.post(url_path, headers=headers, data=payload)
        result = res.json()
        return result
```

Result

{'errorCode': '0', 'status': 'live', 'info': {'orderId': '60066DE5-BD9F-4FCF-8381-F623D4E88101', 'feeRate': '0.001', 'fee': '0', 'remainQty': '1.0000', 'timestamp': '1508945672', 'price': '230', 'currency': 'XRP', 'type': 'bid',

'qty': '1.0000'}, 'result': 'success'}

```
TestCase
    def test_get_my_order_status(self):
        print(inspect.stack()[0][3])
        result = self.coinone_machine.get_my_order_status(coin_type="xrp",
order_id="60066de5-bd9f-4fcf-8381-f623d4e88101")
        assert result
        print(result)
```

마지막 편집일시 : 2017년 10월 28일 9:24

==========

04. DB 서버 구축하기

이번 장부터는 실제 셋업 과정과 구현을 진행한다. 앞서 언급했듯이 MongoDB와 관련된 내용만 해도 책 한 권 분량을 적을 수 있다. 이 프로젝트의 목표에서는 벗어나므로 여기서는 셋업 및 설정, 이 프로젝트에서 사용될 DB Operation, Query에 한정해서 다룰 예정이다. 그리고 Python으로 Wrapper를 만들어서 사용할 예정이다.

마지막 편집일시 : 2017년 10월 8일 4:17

04.1. MongoDB

MongoDB 는 Document-oriented Database로 NoSQL의 대표적인 DB이다. 이 db를 선택하게 된 이유는 schema free, JSON과 궁합이 잘 맞아 사용하게 되었다.

이 프로젝트에서는 MongoDB 3.4 버전을 기준으로 진행한다. DB를 운영한다는 건 쉬운일이 아니다. 설치해서 단순히 혼자 쓰는 경우라면 손십게 사용할 수 있지만 서비스를 운영하게 될 경우 사용자 관리, 자원관리, 성능튜닝, 보안 등 굉장히 전문적인 지식과 스킬을 요한다. 대부분의 상용서비스는 DBA 인력을 따로 고용하여 DB에 대한 관리를 전문적으로 운영한다.

MongoDb는 Document 단위로 데이터를 관리하게 된다. 이 Document는 BSON 타입이며 JSON과 호환하여 사용할 수 있다.

그리고 Collection은 Document의 집합이며 Database는 collection의 집합이다.

PyMongo library는 db console에서 하는 작업들을 python으로도 할 수 있게 해준다 PyMongo를 이용해 MongoDB wrapper 를 만들어서 mongodb을 조금 편하게 하도록 할 것이다.

이번 프로젝트에서는 ORM을 사용하지 않는다. 이번 프로젝트 범위 내에서는 우리가 저장하게 되는 가격정보나 거래정보를 Object로 컨트롤하는 부분이 많지 않아 제외하였다.

필자는 다른 프로젝트에서 Django, Grails의 ORM을 이용해봤지만 굉장히 DB의 item들을 promammatic 하게 정교하게 Control할 수 있어 선호하는 편이지만 Schema가 복잡하지 않

고 DB의 작업 관련 구현 내용이 많지 않으면 ORM을 사용하지 않는 편이다.

마지막 편집일시 : 2017년 10월 8일 1:25

04.1.1. Install

MongoDB의 install 과정은 공식 tutorial을 참고하자
공식
tutorial https://docs.mongodb.com/manual/tutorial/install-mongodb-on-ubuntu/

마지막 편집일시 : 2017년 10월 8일 1:25

04.1.2. Configuration

db의 configuration 설정 변경은 /etc/mongod.conf에서 변경할 수 있다. 우선 MongoDB의 알려진 **port는 27017번인데 27010으로 변경해서 사용하자**.
그리고 현재는 외부에서 접속할 수 없도록 **bindIp를 127.0.0.1로 설정하자.**
mongod.conf 파일을 열고 아래 설정을 추가한다.
net: port: 27010 bindIp: 127.0.0.1
Mongodb restart를 해야 변경사항이 반영된다.
sudo service mongod restart

마지막 편집일시 : 2017년 10월 8일 1:25

04.1.3. MongoDB Shell
아래 명령어로 접속할 수 있다

mongo -u "admin" -p "[password]" --port 27010 --authenticationDatabase "admin"

그림과 같이 Shell이 뜬다면 정상적으로 설치된 것이다.

MongoDB Shell에서 command를 확인하고 실행해서 데이터, Collection을 생성, 조회, 변경, 삭제등 모든 것이 가능하다.

마지막 편집일시 : 2017년 10월 8일 1:25

---

7. 04.1.4. Databases

현재 사용 중인 Database의 list를 확인하기 위해서는 db shell에서 아래 명령어로 확인할 수 있다.

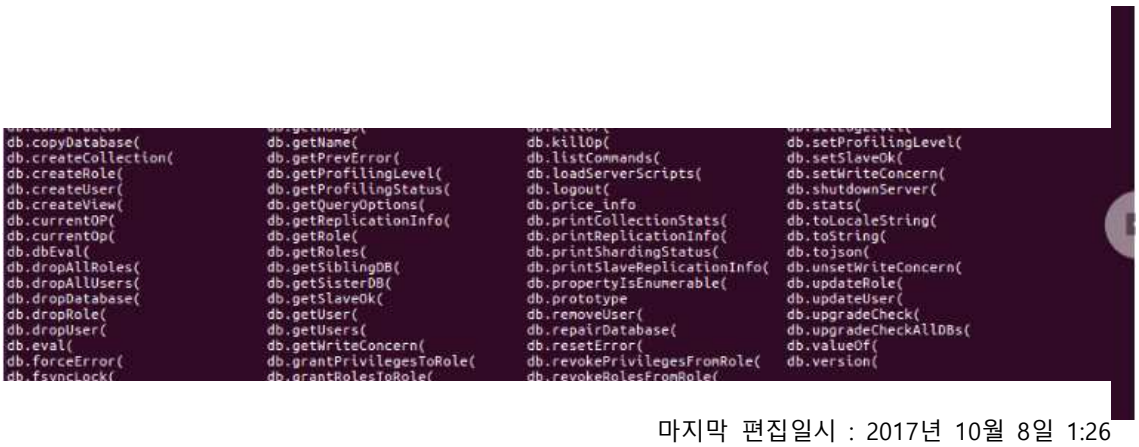
show databases;

database의 전환이나 생성은 use command로 가능하다. 실제로 생성은 collection을 생성할 때 생성된다.

이 프로젝트에서 사용할 Database는 coiner로 하자

use coiner;

db를 선택하고 나면 db. 을 입력한 후 tab을 누르면 그림과 같이 사용 가능한 command list가 출력된다. Assistant 기능이 잘되어있다. 물론 collection을 선택한 이후에도 collection에서 사용가능한 command list를 확인할 수 있다.



마지막 편집일시 : 2017년 10월 8일 1:26

---

8. 04.1.5. Create Collection

Collection은 document의 집합이며 생성시에 여러가지 제약조건을 걸어 둘 수 있다. document가 무한정 늘어나는 것을 방지하며 각 필드에 대한 포맷을 강제할 수도 있다.

자세한 내용은 공식 reference를 참조하자.

공식 reference
https://docs.mongodb.com/manual/reference/method/db.createCollection/

이번 프로젝트에서는 그림과 같이 2개의 database에 4개의 collection을 만들어서 사용한다.

trader	trade_status	현재 나의 주문상태
	trade_history	주문이 Cancel되거나 Complete된 이력정보
	trade_strategy	자동주문시 사용할 parameter

use coiner;
db.createCollection("price_info")
use trader:
db.createCollection("trade_status")
db.createCollection("trade_history")
db.createCollection("trade_strategy")
show command로 정상적으로 생성되었는지 확인해 보자. show collections;
마지막 편집일시 : 2017년 10월 8일 1:26
---

9. 04.1.6. CRUD

MongoDB document의 CRUD작업을 해보자.

Insert: db.collection.insert()

db.price_info.insert({"item1":"123","item2":"btc","item3":123,"item4":new Timestamp()})
db.price_info.insert({"item1":"123","item2":"eth","item3":"123","item4":new Date()})

db.price_info.insert({"item1":"123","item2":"xrp","item3":"123","item4":new Date(),"item5":[1,"2",3]})

MongoDB는 각 documentd의 column간 type이 같을 필요 없다. 그리고 list를 insert할 수도 있다. column간 이름이 같다고 type이 같을 필요도 없다. 이 외에도 insertMany를 이용해 여러 document를 한번에 넣을 수도 있다.

Selet: db.collection.find()

insert한 결과를 찾아보자.

price_info의 모든 document를 찾을 때

db.price_info.find()

price_info의 item2의 값이 btc인 document를 찾을 때

db.price_info.find({"item2”:"btc"})

Update: db.collection.update(query, update, options)

item2가 eth인 항목에 item5값을 update한다. (없으면 추가)

db.price_info.update({"item2":"eth"}, {$set:{"item5":"567"}})

update부분에 $set을 주지 않고 실행하게 되면 item2가 eth인 documentd의 모든 document가 {"item5”:”567”} 로 바뀌게 된다. 이부분은 주의해야 한다.
db.price_info.update({"item2":"eth"}, {"item5":"567"})

Delete: db.collection.deleteMany(), db.collection.ddeleteOne()

item2가 btc인 항목을 전부 지울때

db.price_findo.deleteMany({"item2":"btc"})

item2가 btc인 항목을 하나만 지울때

db.price_findo.deleteOne({"item2":"btc"})

전체 collection을 전부 지울때

db.price_info.deleteMany({})

마지막 편집일시 : 2017년 10월 8일 1:26

---

10. 04.1.7. Aggregation

Aggregation 을 해보자.

MongoDB 의 aggregation 관련 공식 레퍼런스도 참고하자. 설명이 잘 되어 있다.
https://docs.mongodb.com/manual/aggregation/

우리가 구할 것은 coin 별 1 시간 안의 최대, 최소 값이다.

match 에원하는년도,달을입력하고 id 에 키는 group 을묶을 key 를넣어준후 match 에원
하는년도 달을입력하고 id 에 키는 group 을묶을 key 를넣어준후 min, $max 를 이용하여

값을 구하자.

MongoDB 의 aggragation 은 pipeline 개념을 이용하고 있어 기존의 SQL GroupBy 에 익숙해져있다면 처음에 조금 생소하게 느껴질 수 있다.

MongoDB's aggregation framework is modeled on the concept of data processing pipelines. Documents enter a multi-stage pipeline that transforms the documents into an aggregated result. The most basic pipeline stages provide filters that operate

like queries and document transformations that modify the form of the output document.

```
MongoDB Enterprise > db.price_info.aggregate(

  [

    {$match:{month:9,year:2017}},

    {$$group:{ _id:"$coin",

                  min_val:{$min:"$price"},

                  max_val:{$max:"$price"}

      }

     }

   ]

)
```

Result: {"_id" : "eth_krw", "min_val" : "220000", "max_val" : "433500" } { "_id" : "etc_krw", "min_val" : "10000", "max_val" : "9990" } { "_id" : "xrp_krw", "min_val" : "168", "max_val" : "295" } { "_id" : "btc_krw", "min_val" : "3334000", "max_val" : "5499000" }

마지막 편집일시 : 2017년 10월 12일 11:03

---

11. 04.2. PyMongo Wrapper 만들기

04.2. PyMongo Wrapper 만들기

PyMongo library는 MongoDB의 Shell에서 하던 작업을 Python으로 할 수 있도록 도와준다.

이것을 wrapping 해 이 프로젝트에서 DB 작업에 공통적으로 쓸 수 있도록 할 것이다.

최종 구현 후 Directory 구조이다.

```
├── README.md
├── config
|   ├── README.md
|   └── config.ini
├── db
|   └── mongodb
|       └── mongodb_handler.py
├── machine
|   └── korbit_machine.py
└── tests
    ├── db
    |   └── mongodb
    |       └── test_mongodb_handler.py
    └── machine
        └── test_korbit_machine.py
```

마지막 편집일시 : 2017년 10월 8일 4:17

04.2.1. Config

config.ini 파일에 MONGODB Section을 추가하자 remote_host는 서버 호스팅의 서버 url을 넣어주자.

```
[MONGODB]
local_ip = 127.0.0.1
port = 27010
remote_host = hyunny88@cloudapp.net
user = xxxx
password = xxxx
```

마지막 편집일시 : 2017년 10월 12일 3:56

04.2.2. Test

4.2.2에서 구현한 PyMongo wrapping class의 TestCase 이다.

```
"""
Test mongo handler
"""
import unittest, inspect
from db.mongodb.mongodb_handler import MongoDbHandler
```

```
class MongoDbHandlerTestCase(unittest.TestCase):
    """
    MongodbHanlderTestCase
    """
    def setUp(self):
        self.mongodb = MongoDbHandler("local", "stocker_test", "stocks")

    def test_set_db(self):
        """
        test set_db
        """
        print(inspect.stack()[0][3])
        self.mongodb.set_db("stocker_test")
        self.assertEqual(self.mongodb.get_current_db_name(), "stocker_test")

    def test_set_connection(self):
        """
        test set_connections
        """
        print(inspect.stack()[0][3])
        self.mongodb.set_collection("stocks")
        self.assertEqual(self.mongodb.get_current_collection_name(), "stocks")

    def test_get_db_name(self):
        print(inspect.stack()[0][3])
        dbname = self.mongodb.get_current_db_name()
        assert dbname
        print(dbname)

    def test_get_collection_name(self):
        print(inspect.stack()[0][3])
        collection_name = self.mongodb.get_current_collection_name()
        assert collection_name
        print(collection_name)


    def test_insert_item(self):
        """
        test insert_item
        """
```

```
        print(inspect.stack()[0][3])
        id = self.mongodb.insert_item({"code":"0003", "name":"TEST3"})
        print(id)

    def test_find_item(self):
        """
        test find_item
        """
        print(inspect.stack()[0][3])
        item = self.mongodb.find_item({"code":"0003"})
        if item is not None:
            pass
            #self.assertEqual(item["name"], "TEST3")
        print(item)

    def test_insert_items(self):
        """
        test insert_items
        """
        print(inspect.stack()[0][3])
        items = [{"code":"0004", "name":"Test4"},
                 {"code":"0005", "name":"Test5"}]
        ids = self.mongodb.insert_items(items)
        print(ids)

    def test_delete_items(self):
        print(inspect.stack()[0][3])
        items = [{"delete_code":"0004", "name":"Test4"},
                 {"delete_code":"0005", "name":"Test5"}]
        ids = self.mongodb.insert_items(items)
        print(ids)
        self.mongodb.delete_item({})


    def test_move_document(self):
        print(inspect.stack()[0][3])
        items = [{"delete_code":"0006", "name":"Test6"},
                 {"delete_code":"0007", "name":"Test7"}]
        ids = self.mongodb.insert_items(items)
        item_list = self.mongodb.find_item({})
        for item in item_list:
```

```
            self.mongodb.set_collection("stocks_test")
            self.mongodb.insert_item(item)
            self.mongodb.set_collection("stocks")
            self.mongodb.delete_item({"_id":item["_id"]})

    def tearDown(self):
        pass

if __name__ == "__main__":
    unittest.main()
```

마지막 편집일시 : 2017년 10월 8일 4:46

04.2.2. 구현

PyMongo의 메소드를 호출을 한번 Wrapping해주는 역할이다. 기본적으로 04.1.6 CRUD에서 확인한 내용에 대한 구현이다.

```
"""
Mongodb handler
"""
from pymongo import MongoClient
from pymongo.cursor import CursorType
import configparser

class MongoDbHandler:
    """
    mongodb handler
    """
    def __init__(self, mode="local", db_name=None, collection_name=None):
        """
        initializer
        """
        config = configparser.ConfigParser()
        config.read('conf/config.ini')
        self.db_config = {}
        self.db_config["local_ip"] = config['MONGODB']['local_ip']
        self.db_config["port"] = config['MONGODB']['port']
        self.db_config["remote_host"] = config['MONGODB']['remote_host']
        self.db_config["user"] = config['MONGODB']['user']
        self.db_config["password"] = config['MONGODB']['password']
```

```
        if mode == "remote":
            self._client =
MongoClient("mongodb://{user}:{password}@{remote_host}:{port}".format(**self.db_conf
ig))#'localhost', 27017)
        elif mode == "local":
            self._client =
MongoClient("mongodb://{user}:{password}@{local_ip}:{port}".format(**self.db_config))

        if db_name is not None:
            self._db = self._client[db_name]
            if collection_name is not None:
                self._collection = self._db[collection_name]
            else:
                raise Exception("Need to collection name")
        else:
            raise Exception("Need to db name")

    def set_db(self, db_name=None, collection_name=None):
        """
        mongo: use db
        """
        if db_name is None:
            raise Exception("Need to dbname name")

        self._db = self._client[db_name]
        if collection_name is not None:
            self._collection = self._db[collection_name]



    def set_collection(self, collection_name=None):
        """
        change collection
        """
        if collection_name is None:
            raise Exception("Need to dbname name")
        self._collection = self._db[collection_name]

    def get_current_db_name(self):
        return self._db.name#current_db_name

    def get_current_collection_name(self):
```

```
        return self._collection.name#current_collection_name

    def insert_item(self, data, db=None, collection=None):
        """
        insert item one
        """
        if db is not None:
            self._db = self._client[db]
        if collection is not None:
            self._collection = self._db[collection]
        return self._collection.insert_one(data).inserted_id

    def insert_items(self, datas, db=None, collection=None):
        """
        insert many item
        """
        if db is not None:
            self._db = self._client[db]
        if collection is not None:
            self._collection = self._db[collection]
        return self._collection.insert_many(datas).inserted_ids

    def find_item(self, condition=None, db=None, collection=None):
        """
        find_item
        """
        if condition is None:
            condition = {}
        if db is not None:
            self._db = self._client[db]
        if collection is not None:
            self._collection = self._db[collection]
        return self._collection.find(condition, no_cursor_timeout=True,
cursor_type=CursorType.EXHAUST)

    def find_one_item(self, condition=None, db=None, collection=None):
        """
        find_item
        """
        if condition is None:
            condition = {}
```

```
        if db is not None:
            self._db = self._client[db]
        if collection is not None:
            self._collection = self._db[collection]
        return self._collection.find_one(condition)

    def delete_item(self, condition=None, db=None, collection=None):
        """
        delete item
        """
        if condition is None:
            raise Exception("Need to condition")
        if db is not None:
            self._db = self._client[db]
        if collection is not None:
            self._collection = self._db[collection]
        return self._collection.delete_many(condition)

    def update_item(self, condition=None, update_value=None, db=None,
collection=None):
        """
        update item
        """
        if condition is None:
            raise Exception("Need to condition")
        if update_value is None:
            raise Exception("Need to update value")
        if db is not None:
            self._db = self._client[db]
        if collection is not None:
            self._collection = self._db[collection]
        return self._collection.update_many(filter=condition, update=update_value)

    def aggregate(self, pipeline=None, db=None, collection=None):
        if pipeline is None:
            raise Exception("Need to pipeline")
        if db is not None:
            self._db = self._client[db]
        if collection is not None:
            self._collection = self._db[collection]
        return self._collection.aggregate(pipeline)
```

마지막 편집일시 : 2017년 10월 12일 10:08

---

05. BackEnd

5장에서는

- 데이터를 DB에 저장
- 자동거래 로직을 구현
- Slack 메신저와 연동
- Schedule Job
- API 서버

위 5가지 내용을 주제로 글을 작성하고자 한다.

마지막 편집일시 : 2017년 10월 8일 5:04

05.1. 가상화폐 데이터 저장하기

3장과 4장에서 구현한 Korbit, MongoDB module로 가상화폐 데이터를 저장하는 job을 생성할 것이다.

우선 Coiner class는 Korbit을 이용해 tick과, filled_order를 가져오는 부분을 한번 더 wrapping 했다. 이후에 코인원이나 다른 거래소를 이용할 경우 Coiner class에 넘겨주는 거래소 모듈만 코인원 거래소 모듈만 구현하여 넘겨주면 된다.

그리고 이 Coiner class를 이용하여 저장할 데이터를 가져오고 Main에서는 MongoDB에 insert 하는 작업을 진행한다. 우선 Main class로 구현한 뒤에 Schedule Job에서 celery task로 변환해보자.

마지막 편집일시 : 2017년 10월 8일 10:17

05.1.1. Coiner

filled_order를 호출한 결과에 년, 월, 일, 시간정보를 추가적으로 붙여서 넣었다. 추후에 MongoDB Query를 짤때 편의를 위해서 입력하였다. Query로 해도 무방하지만 굉장히 Query가 복잡해진다. 저장공간을 쓰더라도 개발편의성을 위해 별도로 저장하였다.

```
from datetime import datetime
from machine.base_machine import Machine

class Coiner:
    """
    Coiner module class
    """
    def __init__(self, machine=None):
        if machine is None:
            raise Exception("Need to machine object")
        if not isinstance(machine, Machine):
            raise Exception("It's not a Machine subclass")
        self.machine = machine
```

```
        self.machine.set_token()

    def get_ticker(self, currency_pair=None, request_type="detailed"):
        if currency_pair is None:
            return None
        result = self.machine.get_ticker(currency_pair=currency_pair,
request_type=request_type)
        result["coin"] = currency_pair
        result["timestamp"] = result["timestamp"]/1000
        return result

    def get_filled_orders(self, currency_pair=None, time="minute"):
        if currency_pair is None:
            return None
        result = self.machine.get_filled_orders(currency_pair, time)
        for item in result:
            item["coin"] = currency_pair
            item["timestamp"] = item["timestamp"]/1000
            d = datetime.fromtimestamp(item["timestamp"])
            item["year"] = d.year
            item["month"] = d.month
            item["day"] = d.day
            item["hour"] = d.hour
            item["minute"] = d.minute
            item["second"] = d.second
            item["amount"] = float(item["amount"])
        return result
```

마지막 편집일시 : 2017년 10월 8일 10:33

05.1.2. Main

이제 앞에서 구현한 거래소 모듈과, DB 모듈을 이용하여 MongoDB에 저장하는 task를 만들어 보자. 이 부분은 05.4에서 celery job task로 변환하여 스케쥴링 task로 변환할 것이다.

```
import os
import sys
project_dir = os.path.abspath(os.getcwd())
sys.path.append(project_dir)

from db.mongodb import mongodb_handler
import sys, traceback, configparser
```

```
from optparse import OptionParser
from datetime import datetime
from machine.korbit_machine import KorbitMachine
from scheduler.coiner import Coiner

if __name__ == "__main__":
    config = configparser.ConfigParser()
    config.read('config/config.ini')
    client_id = config['KORBIT']['client_id']
    client_secret = config['KORBIT']['client_secret']
    username = config['KORBIT']['username']
    password = config['KORBIT']['password']
    korbit_machine = KorbitMachine(mode="Prod",client_id=client_id,
                                   client_secret=client_secret,
                                   username=username,
                                   password=password)
    coiner = Coiner(korbit_machine)
    result_etc = coiner.get_filled_orders(currency_pair="etc_krw")
    result_eth = coiner.get_filled_orders(currency_pair="eth_krw")
    result_btc = coiner.get_filled_orders(currency_pair="btc_krw")
    result_xrp = coiner.get_filled_orders(currency_pair="xrp_krw")
    mongodb = mongodb_handler.MongoDbHandler("local", "coiner", "price_info")
    result_list = result_etc + result_eth + result_btc + result_xrp
    ids = mongodb.insert_items(result_list)
```

마지막 편집일시 : 2017년 10월 9일 7:33

05.2. Schedule Job

Schedule Job은 주기적으로 동작하면서 필요한 작업을 수행한다. Schedule Job을 수행하는 방법에는 여러가지가 있지만 여기서는 Python의 Celery를 이용하여 Python job을 주기적으로 실행하는 방법에 대해서 작성할 것이다.

마지막 편집일시 : 2017년 10월 9일 7:39

05.2.1. Redis

https://redis.io/

우선 Redis가 설치해야 한다. 공식 홈페이지에 들어가 보면 Redis는 in-memory data structure store로 DB, cache, message broker등으로 다양하게 이용할 수 있다. Celery를 사용하기 위해서는 message broker가 필요한데 RabbitMQ, ZeroMQ등 오픈소스 Message broker를 이용할 수도 있지만 Redis를 활용할 수도 있다. 우리는 Redis를 사용하여 Celery를 구축해 보자.

마지막 편집일시 : 2017년 10월 9일 7:39

12. 05.2.1.1 Install

공식 홈페이지의 Download를 들어가보면 Installation에서 설치방법을 확인할 수 있다.

https://redis.io/download

다운로드 받은 후 make를 실행한 후 src의 redis-server를 실행하면 된다.

wgethttp://download.redis.io/releases/redis－4.0.2.tar.gzwgethttp://download.redis.io/releases/redis－4.0.2.tar.gz tar xzf redis－4.0.2.tar.gz cdredis－4.0.2cdredis－4.0.2 make

$ src/redis-server

redis-server를 실행시킨 후 아래와 같이 화면이 뜨면 정상적으로 서버가 뜬 상태이다.

redis-server로 실행 후 다른 터미널을 띄워서 redis-cli로 접속해서 아래 명령어로 정상 동작하는지 확인해 보자.

$ src/redis-cli

redis> set foo bar

redis> get foo "bar"

ping을 입력해보면 PONG으로 response가 온다면 서버 설치가 완료되었다.

마지막 편집일시 : 2017년 10월 9일 7:54

05.2.2. Celery

http://www.celeryproject.org/
Celery 공식페이지를 들어가보면 소개 및 Tutorials을 확인할 수 있다.
Celery는 비동기 task job을 queue하는 framework이다. Celery는 실시간 처리에 초점이 맞춰져있지만 우리가 하려고 하는 scheduling 또한 잘 지원한다.

마지막 편집일시 : 2017년 10월 9일 7:54

13. 05.2.2.1. Install

설치방법은 간단하다. pip로 설치하자.

```
$ pip install Celery
```

그리고 celery를 실행해보면

```
---- -- - - ---- Commands- -------------- --- ------------

+ Main:
|    celery worker
|    celery events
|    celery beat
|    celery shell
|    celery multi
|    celery amqp

+ Remote Control:
|    celery status

|    celery inspect --help
|    celery inspect active
|    celery inspect active_queues
|    celery inspect clock
|    celery inspect conf [include_defaults=False]
|    celery inspect memdump [n_samples=10]
|    celery inspect memsample
|    celery inspect objgraph [object_type=Request] [num=200 [max_depth=10]]
|    celery inspect ping
|    celery inspect query_task [id1 [id2 [... [idN]]]]
|    celery inspect registered [attr1 [attr2 [... [attrN]]]]
|    celery inspect report
|    celery inspect reserved
|    celery inspect revoked
|    celery inspect scheduled
|    celery inspect stats

|    celery control --help
|    celery control add_consumer <queue> [exchange [type [routing_key]]]
|    celery control autoscale [max [min]]
|    celery control cancel_consumer <queue>
|    celery control disable_events
|    celery control election
|    celery control enable_events
|    celery control heartbeat
|    celery control pool_grow [N=1]
|    celery control pool_restart
|    celery control pool_shrink [N=1]
|    celery control rate_limit <task_name> <rate_limit (e.g., 5/s | 5/m | 5/h)>
|    celery control revoke [id1 [id2 [... [idN]]]]
|    celery control shutdown
|    celery control terminate <signal> [id1 [id2 [... [idN]]]]
|    celery control time_limit <task_name> <soft_secs> [hard_secs]

+ Utils:
|    celery purge
|    celery list
|    celery call
|    celery result
|    celery migrate
|    celery graph
|    celery upgrade

+ Debugging:
|    celery report
|    celery logtool
---- -- - - --------- -- - -------------- --- ------------

Type 'celery <command> --help' for help using a specific command.
```

사용가능한 command를 보여준다. 여기까지 진행되었으면 성공적으로 설치가 완료된 것이다.

추가적으로 celery에 Redis를 broker로 사용하기 위해서는 celery[redis]를 설치해야 한다.

$ pip install celery[redis]

마지막 편집일시 : 2017년 10월 9일 8:09

05.2.2.2. Task

5.1.2 Main에서 작성한 scheduler/coin_price.py의 코드를 약간 수정하자.

main으로 작성한 부분을 function으로 수정하고 아래 부분을 추가하면 된다.

```
from celery import Celery
app = Celery('get_coin_info', backend='redis://localhost:6379/0',
broker='redis://localhost:6379/0')
```

function으로 추가한 부분에는 @app.task decorator를 붙여서 이 function이 celery의 task임을 알려줘야 한다.

```
import os
import sys
project_dir = os.path.abspath(os.getcwd())
sys.path.append(project_dir)

from db.mongodb import mongodb_handler
import sys, traceback, configparser
from optparse import OptionParser
from datetime import datetime
from machine.korbit_machine import KorbitMachine
from scheduler.coiner import Coiner
from celery import Celery

app = Celery('get_coin_info', backend='redis://localhost:6379/0',
broker='redis://localhost:6379/0')

@app.task
def get_coin_info():
    config = configparser.ConfigParser()
    config.read('config/config.ini')
    client_id = config['KORBIT']['client_id']
    client_secret = config['KORBIT']['client_secret']
    username = config['KORBIT']['username']
    password = config['KORBIT']['password']
    korbit_machine = KorbitMachine(mode="Prod",client_id=client_id,
```

```
                                        client_secret=client_secret,
                                        username=username,
                                        password=password)
    coiner = Coiner(korbit_machine)
    result_etc = coiner.get_filled_orders(currency_pair="etc_krw")
    result_eth = coiner.get_filled_orders(currency_pair="eth_krw")
    result_btc = coiner.get_filled_orders(currency_pair="btc_krw")
    result_xrp = coiner.get_filled_orders(currency_pair="xrp_krw")
    mongodb = mongodb_handler.MongoDbHandler("local", "coiner", "price_info")
    result_list = result_etc + result_eth + result_btc + result_xrp
    ids = mongodb.insert_items(result_list)
```

마지막 편집일시 : 2017년 10월 9일 8:24

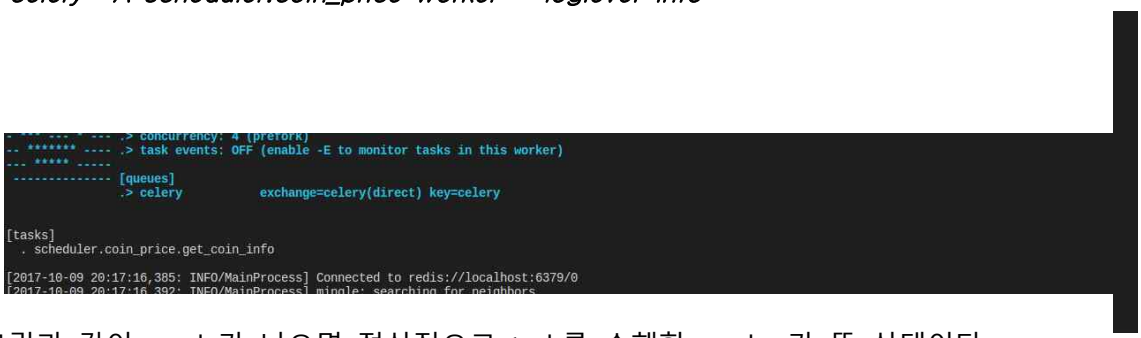
---

14. 05.2.2.3. Schedule Job

이번 장에서는 우선 scheduler job이 아닌 celery로 task를 띄우고 수행하는 방법을 우선적으로 해본 후에 schedule job으로 변경해보자.

우선 redis 서버가 정상적으로 동작하고 있는 상태에서 아래 command를 수행해 보자.

$ celery -A scheduler.coin_price worker --loglevel=info



그림과 같이 ready가 나오면 정상적으로 task를 수행할 worker가 뜬 상태이다.

여기서 python을 실행한 후 get_coin_info를 import한 후 delay()로 실행해보자.

Python 3.5.2 (default, Nov 17 2016, 17:05:23)

```
[GCC 5.4.0 20160609] on linux

Type "help", "copyright", "credits" or "license" for more information.

>>> from scheduler.coin_price import get_coin_info

>>> get_coin_info.delay()

<AsyncResult: c57f71e9-a1d2-4e33-8861-3c2e5baac565>

>>>
```

delay()를 수행하고 나면 그림과 같이 celery가 task를 받았다고 나오면 수행결과가 successed라는 message와 함께 수행시간을 표시해 준다.

여기까지는 celery를 이용해 비동기로 task를 수행하는 방법까지 해보았고 우리가 하고자 하는 **schedule job을 구현해보자**.

다시 **coin_price.py를 열어서** 아래 내용을 추가하자.

```
app.conf.beat_schedule = {

    'add-every-1-min': {

        'task': 'scheduler.coin_price.get_coin_info',

        'schedule': 60.0,
```

```
            'args':()

        },

}
```

그리고 celery를 실행할때 아래와 같이 -B 옵션을 주도록 하자. 그러면 따로 python을 띄워서 task를 trigger하지 않아도 60초간격으로 수행되는 것을 볼 수 있다. 숫자가 아닌 crontab 표현식을 쓸 수도 있다. 자세한 내용은 아래 레퍼런스를 참고 바란다.

http://docs.celeryproject.org/en/latest/userguide/periodic-tasks.html

$ celery -A scheduler.coinprice worker -B --loglevel=info

그러나 여기서 -B 옵션을 주고 실행하는 것은 아래와 같이 공식문서에서 나와있듯이 production에서는 추천하지 않는 다고 나와있다. 보통 production에서는 worker를 여러개 띄우기 때문이다.

You can also embed beat inside the worker by enabling the workers -B option, this is convenient if you'll never run more than one worker node, but it's not commonly used and for that reason isn't recommended for production use:

아래와 같이 task의 last run에 대한 정보를 local database file로 저장해두고 사용하라고 권고하고 있다.

$ celery -A proj beat -s /home/celery/var/run/celerybeat-schedule

Beat needs to store the last run times of the tasks in a local database file (named celerybeat-schedule by default), so it needs access to write in the current directory, or alternatively you can specify a custom location for this file:

Celery Process는 추후에 05.6에서 Supervisord으로 daemon형태로 구동할 것이다.

마지막 편집일시 : 2017년 11월 4일 11:19

---

15. 05.2.2.4. Flower

http://flower.readthedocs.io/en/latest/

Flower는 celery의 모니터링 툴이다. Query나 Task의 상태를 확인해볼 수 있기 때문에 Celery를 사용할때 사용하게 될 일이 생기게 된다.

$ pip install flower

실행법은 간단하다. flower를 실행시키거나 celery로 수행시키면 된다.

flower－Aproj－－port=5555ORflower－Aproj－－port=5555OR celery flower －A proj －－address=127.0.0.1 －－port=5555

```
 'celery.backend_cleanup',
 'celery.chain',
 'celery.chord',
 'celery.chord_unlock',
 'celery.chunks',
 'celery.group',
 'celery.map',
 'celery.starmap',
 'scheduler.coin_price.get_coin_info']
```

실행하고 나면

http://localhost:5555/ web을 알려주는데

들어가보면

모니터링 웹 화면을 볼 수 있다. worker별 상태, process의 성공, 실패 여부등 여러가지를 확인할 수 있고 현재 queue의 상태등을 확인할 수 있다.

flower process 도 05.6에서 Supervisord를 이용하여 수행할 것이다.

마지막 편집일시 : 2017년 11월 4일 11:19

05.2.3. Crontab

전통적으로 주기적인 Job을 실행 시키기 위한 방법은 Crontab이 있다. Cron 표현식을 이용해 Crontab에 등록하게 되면 해당 주기 간격으로 Job이 수행된다.

sys.argv를 이용하면 command line으로 부터 arguments를 받을 수 있고 그 argument로 프로그램을 동작하게하면 crontab으로 하나의 로직을 가지고 여러 코인에 적용할 수 있다.

arguments받기

```
if len(sys.argv) > 0:

        trader = StepTrade(machine=korbit_machine, db_handler=mongodb, strategy=sys.argv[1], coin_type=sys.argv[2], pusher=pusher)
```

trader.run()

실행script

/usr/local/bin/python3 -m strategy.step_trade step_trade_xrp xrp

Crontab

*/5 * * * * ....스크립트경로/step_trade.sh >> 로그경로/step_trade.log 2>&1

마지막 편집일시 : 2017년 11월 4일 6:5

05.3. Push 알람

Buy, Sell, Cancel 등 거래가 일어날때는 Push 알람을 받기 위해 Pusher를 구현하고자 한다. 이 프로젝트에서는 Slack 메신저를 이용할 예정이지만, 텔레그램등 다양한 메신저와 연동할 수 있도록 설계하자.

마지막 편집일시 : 2017년 10월 8일 7:05

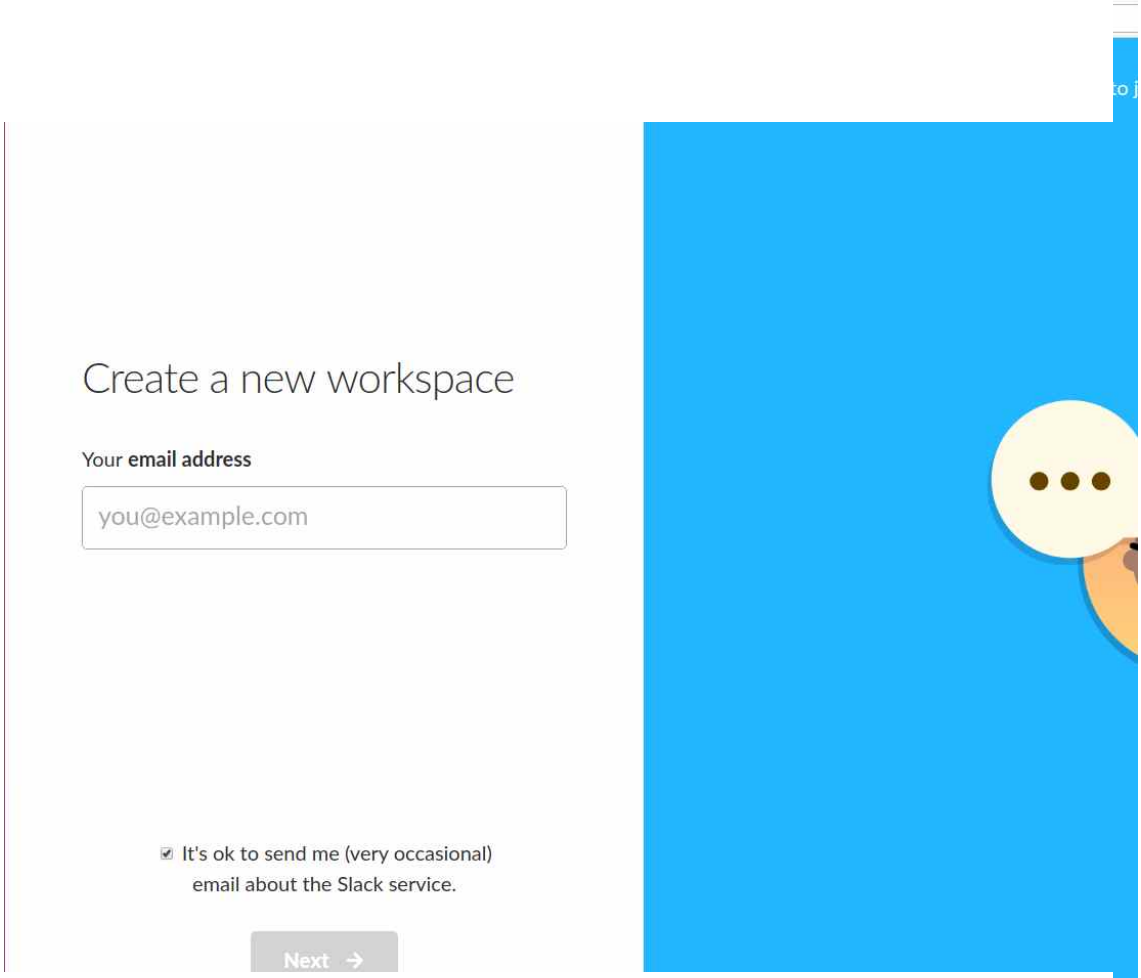
05.3.1. Slack

https://api.slack.com 슬랙 API를 사용하기 위해서는 위 사이트에서 자신의 workplace에 app을 만들고 api key를 발급 받아야한다. 그리고 App store에서 slack app을 검색해서 설치하자.
추후에 다른 메신저와 연동할 수 있도록 고려하여 구현은 abstrace class를 만들고 상속받아 구현한다.
Slack을 이용하면 gitlab, github 등과도 연동이 가능하고 활용처가 많다.

마지막 편집일시 : 2017년 10월 8일 7:05

16. 05.3.1.1. Token 발급

https://slack.com/create#create



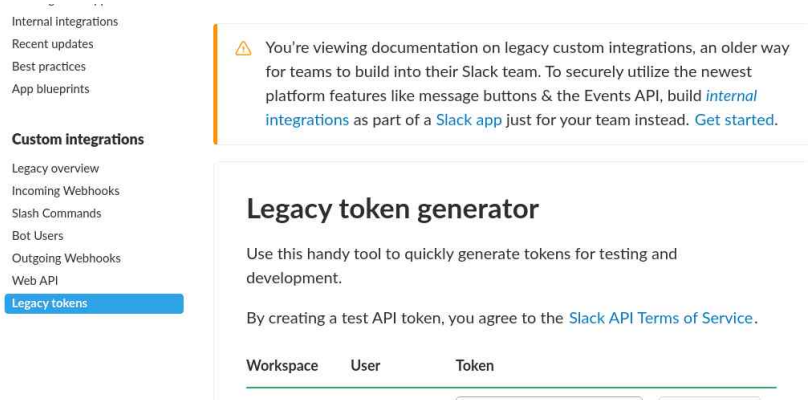
여기서 workspace를 만들자. 용어 그대로 slack 내에 나의 작업공간을 만드는 것이다. 채팅방보다는 상위 개념이다. 이메일로 코드를 받아서 입력하고 부가적인 정보를 입력하면 간단하게 workspace를 만들 수 있다.

이 프로젝트에서 workspace는 https://the-trader-group.slack.com 로 발급받았다.

발급받은 workpace로 들어가보면 개설된 workspace 를 둘러볼 수 있다.

그리고 다시 http://api.slack.com 으로 가서

Legacy tokens에 가면 해당하는 Workspace에 Create Token 버튼을 누르면 Token값을 받을 수 있다.



마지막 편집일시 : 2017년 10월 8일 7:05

05.3.1.2. Install slacker

slacker를 설치하자. slacker는 python으로 slak의 API를 사용할 수 있게 해주는 library이다.

$ pip install slacker

마지막 편집일시 : 2017년 10월 10일 8:53

05.3.1.3. Config

5.3.1.1에서 발급받은 Token을 config.ini에 추가하자

[SLACK] token =

마지막 편집일시 : 2017년 10월 8일 7:05

05.3.1.4. Test

import unittest from pusher.slack import PushSlack

class TestSlacke(unittest.TestCase):

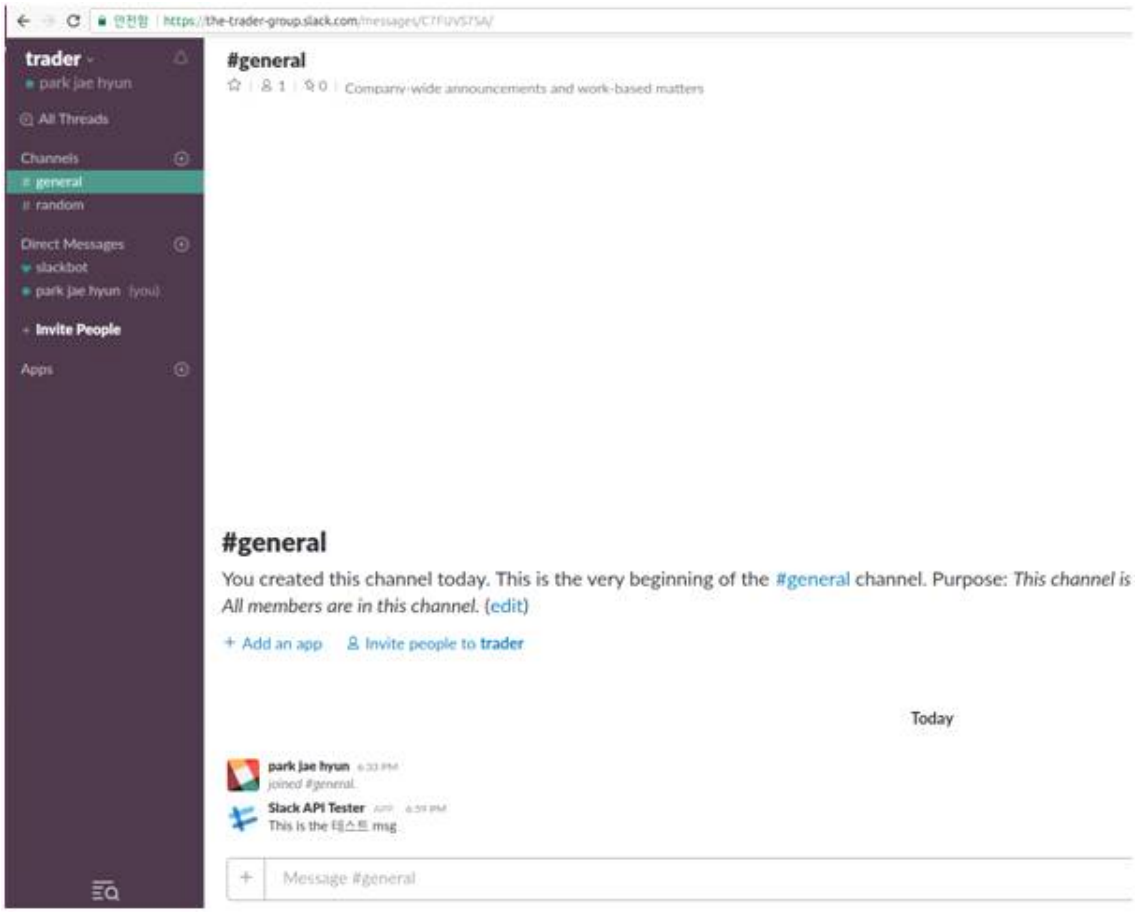
def setUp(self): self.pusher = PushSlack()

def test_send_message(self): self.pusher.send_message("#general", "This is the 테스트 msg")

def tearDown(self): pass

if name == "main": unittest.main()

테스트를 수행해보면 push가 정상적으로 오는 것을 확인할 수 있다. 물론 모바일에서도 정상적으로 알람을 받을 수 있다.



마지막 편집일시 : 2017년 10월 8일 7:07

05.3.1.4. 구현

굉장히 단순하기 때문에 크게 어려운 부분은 없이 구현할 수 있다.

from abc import ABC, abstractmethod

class Pusher(ABC): @abstractmethod def send_message(self, thread, message): pass

from slacker import Slacker from pusher.base_pusher import Pusher import configparser

class PushSlack(Pusher):

```
def __init__(self):
    config = configparser.ConfigParser()
    config.read('conf/config.ini')
    api_key = config['SLACK']['api_key']
    self.slack = Slacker(api_key)

def send_message(self, thread=None, message=None):
    self.slack.chat.post_message(thread, message)
```

마지막 편집일시 : 2017년 10월 12일 3:5

05.4. Trading 로직 구현하기

여기까지 왔으면 Trading 로직 구현에 필요한 사항은 대부분 갖춰졌다.
아래 3가지를 이용하여 Trading 전략을 세우고 구현 해보자.

- 거래소 거래 모듈
- 코인 가격정보
- 알람

Trading 로직 구현은 필자가 운영하고 있는 로직은 작성하지 않겠다. 전체적으로 Trading의 한 사이클만 작성하겠다. 추가적인 부분은 독자 스스로 로직, 전략을 세우고 잘 운영해보도록 하자.

마지막 편집일시 : 2017년 10월 10일 9:01

05.4.1. Trading 전략

필자는 시스템 개발자이지 투자 전문가도 아니고 재테크를 잘 하지도 못하는 사람이다. 다만 어느 날 가상화폐를 알게 되면서 시스템화해보고 싶은 생각에 이 프로젝트를 진행하였다. 그래서 필자가 세우는 Trading 전략은 근본 없는 전략일 수 있다. 수학적이거나, 예측모델을 적용한다거나, Trading 전략을 세우는 부분은 이 글을 읽는 독자가 더 잘할 수 있을 거라 생각한다. 실제로 현재 운영하고 있는 코드를 설명하고자 한다.
필자는 우선 여기서 구현할 전략은 아래와 같다.
1) 최근 1시간의 코인 거래정보에서 최저, 최대값, 마지막 체결가를 가지고 온다. 2) min, max, 거래량 및 최저, 최대 마지막 체결가를 활용하여 상승세, 하락장을 구분한다. 3) 하락장일 경우 - 매수 금지 4) 상승장, 변동폭이 작은 경우 bid 가격을 단계별로 구분하여 주문하도록 한다.(동일 가격범위내는 bid 금지) 5) 거래 상태 관리 BUY_ORDERED: BUY 주문 상태 BUY_COMPLETED: BUY 주문 체결 상태, SELL_ORDERED로 상태를 넘김 SELL_ORDERED: SELL 주문 상태 SELL_COMPLETED: SELL 주문 체결 상태

마지막 편집일시 : 2017년 11월 5일 7:12

05.4.2. 구현 &Schedule Job

이때까지 만들어 둔 모듈을 이용하여 전체 trading 로직을 구현하였다.
AbstractClass에는 전략

```
class Strategy(ABC):
    @abstractmethod
    def run(self):
        pass

    def update_trade_status(self, db_handler=None, buy=None, value=None):
        if db_handler is None or buy is None or value is None:
            raise Exception("Need to buy value or status")
        db_handler.set_db("trader","trade_status")
        db_handler.update_item(buy, {"$set":value})

    def order_buy_transaction(self, machine=None, db_handler=None,
coin_type=None, item=None, order_type="limit"):
        if coin_type is None or item is None:
            raise Exception("Need to param")
        db_handler.set_db("trader","trade_status")
        result = machine.buy_coin_order(coin_type=coin_type,
                                        price=item["buy"],

qty=item["buy_amount"],#str(self.BUY_COUNT),
                                        order_type=order_type)
        if result["status"] == "success":
            db_handler.insert_item({"status":"BUY_ORDERED",
                                    "coin":coin_type,
                                    "buy_order_id":str(result["orderId"]),
                                    "buy_amount":str(item["buy_amount"]),
                                    "buy":str(item["buy"]),

"buy_order_time":int(datetime.datetime.now().timestamp()),
                                    "desired_value":str(item["desired_value"]),
                                    "transaction_status":"success",
                                    "machine":str(machine)})

    def order_sell_transaction(self, machine=None, db_handler=None,
coin_type=None, item=None, order_type="limit"):
        if coin_type is None or item is None:
```

```
            raise Exception("Need to param")
        db_handler.set_db("trader","trade_status")
        result = machine.sell_coin_order(coin_type=coin_type,
                                         price=item["desired_value"],
                                         qty=item["real_buy_amount"],
                                         order_type=order_type)
        if result["status"] == "success":
            db_handler.update_item({"_id":item["_id"]},
                        {"$set":{"status":"SELL_ORDERED",
                        "desired_value":str(item["desired_value"]),
                        "sell_order_id":str(result["orderId"]),
                        "error":"success"}
                        })
        else:
            db_handler.update_item({"_id":item["_id"]},{"error": "failed"})

    def order_cancel_transaction(self, machine=None, db_handler=None,
coin_type=None, item=None):
        db_handler.set_db("trader","trade_status")
        if coin_type is None or item is None or type is None:
            raise Exception("Need to param")
        if item["status"] == "BUY_ORDERED":
            result = machine.cancel_coin_order(coin_type, item["buy_order_id"])
        elif item["status"] == "SELL_ORDERED":
            result = machine.cancel_coin_order(coin_type, item["sell_order_id"])
            db_handler.update_item({"_id":item["_id"]},
{"$set":{"status":"CANCEL_ORDERED",
"cancel_order_time":int(datetime.datetime.now().timestamp()),
                                        "error":"success"}})
from strategy.base_strategy import Strategy
from machine.korbit_machine import KorbitMachine
from machine.coinone_machine import CoinOneMachine
from db.mongodb.mongodb_handler import MongoDbHandler
from pusher.slack import PushSlack
import configparser, datetime, traceback, sys
from logger import get_logger

logger = get_logger("step_trade")

class StepTrade(Strategy):
```

```
    def __init__(self, machine=None, db_handler=None, strategy=None, coin_type=None, pusher=None):
        if machine is None or db_handler is None or coin_type is None or strategy is None:
            raise Exception("Need to machine, db, coin type, strategy")
        if isinstance(machine, KorbitMachine):
            logger.info("Korbit machine")
            self.coin_type=coin_type+"_krw"
        elif isinstance(machine, CoinOneMachine):
            logger.info("CoinOne machine")
        self.machine = machine
        self.pusher = pusher
        self.db_handler = db_handler
        self.machine.set_token()
        result = self.db_handler.find_item({"name":strategy},"trader","trade_strategy")
        self.params = result[0]
        logger.info(self.coin_type)

    def scenario(self):
        last = self.machine.get_ticker(self.coin_type)
        #self.machine.get_list_my_orders(self.coin_type)
        self.last_val=int(last["last"])
        now = datetime.datetime.now()
        one_hour_ago = now - datetime.timedelta(minutes=60)
        one_hour_ago_timestamp = int(one_hour_ago.timestamp())
        pipeline = [{"$match":{"timestamp":{"$gt":one_hour_ago_timestamp}, "coin":self.coin_type}},
                    {"$group":{"_id":"$coin",
                               "min_val":{"$min":"$price"},
                               "max_val":{"$max":"$price"},
                               "sum_val":{"$sum":"$amount"}
                    }}]

        query_result = self.db_handler.aggregate(pipeline,"coiner","price_info")
        for item in query_result:
            hour_max_val = int(item["max_val"])
            hour_min_val = int(item["min_val"])
            hour_sum_val = int(item["sum_val"])
            hour_gap = hour_max_val - hour_min_val
        logger.info('last:'+str(self.last_val))
        logger.info('h_max:'+str(hour_max_val))
```

```
        logger.info('h_min:'+str(hour_min_val))
        logger.info('h_sum:'+str(hour_sum_val))
        logger.info('h_gap:'+str(hour_gap))

        one_day_ago = now - datetime.timedelta(minutes=3600)
        one_day_ago_timestamp = int(one_day_ago.timestamp())
        pipeline_amount = [{"$match":{"timestamp":{"$gt":one_day_ago_timestamp},
"coin":self.coin_type}},
                            {"$group":{"_id":"$coin",
                                       "min_val":{"$min":"$price"},
                                       "max_val":{"$max":"$price"},
                                       "sum_val":{"$sum":"$amount"}
                    }}]

        day_query_result = self.db_handler.aggregate(pipeline_amount)

        for item in day_query_result:
            day_max_val = int(item["max_val"])
            day_min_val = int(item["min_val"])
            day_sum_val = int(item["sum_val"])
            day_sum_avg_val = int(item["sum_val"])/24
            day_gap = day_max_val - day_min_val

        logger.info('last:'+str(self.last_val))
        logger.info('d_max:'+str(day_max_val))
        logger.info('d_min:'+str(day_min_val))
        logger.info('d_sum:'+str(day_sum_val))
        logger.info('d_avg:'+str(day_sum_avg_val))
        logger.info('d_gap:'+str(day_gap))

        "Buy logic part"
        send_msg ={"last":self.last_val, "d_max":day_max_val, "d_min":day_min_val,
                    "d_avg":day_sum_avg_val, "d_gap":day_gap,
"h_max":hour_max_val, "h_min":hour_min_val,
                    "h_sum":hour_sum_val, "h_gap:":hour_gap}
        "if down stream dont try buy"
        if hour_gap > int(self.params["step_value"]) and self.last_val <=
hour_min_val and hour_sum_val > day_sum_avg_val: #down stream
            self.pusher.send_message("#push", "down stream:"+str(send_msg))
            return
        "if up stream try buy"
```

```
        if hour_gap > int(self.params["step_value"]) and self.last_val >=hour_max_val
and hour_sum_val > day_sum_avg_val  : #up stream
            self.pusher.send_message("#push", "up stream"+str(send_msg))
        logger.info("Normal stream")
        logger.info("buy_price:"+str(self.last_val))
        my_orders = self.db_handler.find_item({"coin":self.coin_type,
"$or":[{"status":"BUY_ORDERED"},{"status":"SELL_ORDERED"}],
                                                "buy":{"$gt":
str(self.last_val-int(self.params["step_value"])),
                                                "$lte":
str(self.last_val+int(self.params["step_value"]))}},"trader","trade_status")
        "Check duplicate price bewteen instep_value"
        if my_orders.count() > 0:
            logger.info("Exists order in same price")
            for order in my_orders:
                logger.info("order:"+str(order))
        else:
            logger.info("Buy Order")
            self.item={"buy":str(self.last_val), "buy_amount":self.params["buy_amount"],
"coin":self.coin_type}
            self.item["desired_value"] =
str(int(self.last_val+int(self.params["target_profit"])/float(self.params["buy_amount"])))
            logger.info(self.item)
            wallet = self.machine.get_wallet_status()
            if int(wallet["krw"]["available"]) >
int(self.last_val*float(self.params["buy_amount"])):
                self.order_buy_transaction(machine=self.machine,
                                           db_handler=self.db_handler,
                                           coin_type=self.coin_type,
                                           item=self.item)
                self.pusher.send_message("#push", "buy_ordered"+str(self.item))
            else:
                logger.info("krw is shor")

    def check_my_order(self):
        #Check buy ordered
        buy_orders =
self.db_handler.find_item({"coin":self.coin_type,"status":"BUY_ORDERED"}, "trader",
"trade_status")

        for item in buy_orders:
```

```
            logger.info(item)
            order_result = self.machine.get_my_order_status(self.coin_type, order_id=item["buy_order_id"])
            logger.info(order_result)
            if len(order_result)>0 and order_result[0]["status"]=="filled" and order_result[0]["price"] == item["buy"]:
                order_result_dict = order_result[0]
                real_buy_amount = str(float(order_result_dict["filled_amount"])-float(order_result_dict["fee"]))
                real_buy_value = str(order_result_dict["avg_price"])
                completed_time = int(order_result_dict["last_filled_at"]/1000)
                fee = str(order_result_dict["fee"])
                if order_result_dict["side"] == "bid":
                    self.update_trade_status(db_handler=self.db_handler,
                                             buy={"_id":item["_id"]},
                                             value={"status":"BUY_COMPLETED",
                                              "real_buy_amount":real_buy_amount,
                                              "buy_completed_time":completed_time,
                                              "real_buy_value":real_buy_value,
                                              "buy_fee":fee,
                                              "error":"success"})
                    self.pusher.send_message("#push", "buy_completed:"+str(item))
                elif order_result[0]["status"] =="unfilled" and order_result[0]["price"] == item["buy"]:
                    if int(item["buy"]+int(self.params["step_value"])) < self.last_val:
                        #Cancel Order
                        try:
                            self.order_cancel_transaction(machine=self.machine, db_handler=self.db_handler, coin=self.coin, item=item)
                        except:
                            error = traceback.format_exc()
                            self.update_trade_status(db_handler=self.db_handler,
                                                     buy={"_id":item["_id"]},
                                                     value={"error": "failed"})
                            self.pusher.send_message("#push", "err cancle:"+str(item))

        buy_completed = self.db_handler.find_item({"coin":self.coin_type, "status":"BUY_COMPLETED"})

        for item in buy_completed:
            try:
```

```
                self.order_sell_transaction(machine=self.machine,
db_handler=self.db_handler, coin_type=self.coin_type, item=item)
                self.pusher.send_message("#push", "sell_ordered:"+str(item))
            except:
                error = traceback.format_exc()
                self.update_trade_status(db_handler=self.db_handler,
                                         buy={"_id":item["_id"]},
                                         value={"error":"failed"})

        #Check sell ordered
        sell_orders                                                                    =
self.db_handler.find_item({"coin":self.coin_type,"status":"SELL_ORDERED"})
        for item in sell_orders:
            logger.info(item)
            if "sell_order_id" in item:
                order_result     =     self.machine.get_my_order_status(self.coin_type,
item["sell_order_id"])
                if order_result is not None:
                    logger.info(order_result)
            else:
                continue
            if len(order_result) > 0 and order_result[0]["status"]=="filled" and
order_result[0]["price"] == item["desired_value"]:
                order_result_dict = order_result[0]
                real_sell_amount = str(float(order_result_dict["filled_amount"]))
                real_sell_value = str(order_result_dict["avg_price"])
                completed_time = int(order_result_dict["last_filled_at"]/1000)
                fee = order_result_dict["fee"]
                if order_result_dict["side"] == "ask":
                    self.update_trade_status(db_handler=self.db_handler,
                                             buy={"_id":item["_id"]},
                                             value={"status":"SELL_COMPLETED",
                                             "real_sell_amount":real_sell_amount,
                                             "sell_completed_time":completed_time,
                                             "real_sell_value":real_sell_value,
                                             "sell_fee":fee})
                    self.pusher.send_message("#push", "sell_completed:"+str(item))
            elif len(order_result) > 0 and order_result[0]["status"] =="unfilled" and
order_result[0]["price"] == item["desired_value"]:
                if int(item["desired_value"]) > self.last_val*1.20:
                    self.order_cancel_transaction(machine=self.machine,
```

```
db_handler=self.db_handler, coin_type=self.coin, item=item)
                        self.update_trade_status(db_handler=self.db_handler,
                                                 buy={"_id":item["_id"]},
                                                 value={"status":"KEEP_ORDERED"})
                        self.pusher.send_message("#push", "keeped:"+str(item))

        sell_completed = self.db_handler.find_item({"coin":self.coin_type,
"status":"SELL_COMPLETED"})
        for item in sell_completed:
            self.db_handler.insert_item(item, "trader", "trade_history")
            self.db_handler.delete_item({"_id":item["_id"]}, "trader", "trade_status")

        keeped_orders = self.db_handler.find_item({"coin":self.coin_type,
"status":"KEEP_ORDERED"}, "trader", "trade_status")
        for item in keeped_orders:
            if int(item["desired_value"])*0.9 < self.last_val:
                self.order_sell_transaction(machine=self.machine,
db_handler=self.db_handler, coin=self.coin, item=item)
                self.pusher.send_message("#push", "keep_sell_ordered:"+str(item))
                logger.info("sell order from keeped"+str(item["_id"]))

    def run(self):
        if self.params["is_active"]=="active":
            self.scenario()
            self.check_my_order()
        else:
            logger.info("inactive")

if __name__ == "__main__":
    config = configparser.ConfigParser()
    config.read('conf/config.ini')
    client_id = config['KORBIT']['client_id']
    client_secret = config['KORBIT']['client_secret']
    username = config['KORBIT']['username']
    password = config['KORBIT']['password']
    mongodb = MongoDbHandler("remote", "coiner", "price_info")

    korbit_machine = KorbitMachine(mode="Prod",client_id=client_id,
                                   client_secret=client_secret,
                                   username=username,
                                   password=password)
```

```
    pusher = PushSlack()
    if len(sys.argv) > 0:
        trader = StepTrade(machine=korbit_machine, db_handler=mongodb, strategy=sys.argv[1], coin_type=sys.argv[2], pusher=pusher)
        trader.run()
```

마지막 편집일시 : 2017년 11월 5일 7:05

05.4.3. API Call count 개선하기

Celery 의 message broker로 redis를 설치했지만 redis는 원래 key, value를 저장하는 datastore이다. 본연의 기능을 사용하여 set_token call하는 count를 줄여보자 왜냐하면 공식 문서에도 나와있지만 token생성이나 refresh하는 call은 60분당 60번 call로 제한된다. 그리고 Ticker나 다른 API 도 제한이 걸려있다. 이를 위반하면 대략 30분정도 API가 막힌다.

우리가 구현한 방법으로는 schedule돌때마다 5개의 코인을 돌리게되면 5분 주기밖에 구동할 수 없다. 그러면 Access_token를 redis에 저장해두고 시간이 다 되어갈때 다시 access token을 사용하게 좋겠다.

redis에서 get은 key를 가져오는 것이고. set은 key를 설정하는 command이다. set할때 시간을 주게되면 아래 예시로 3500 초 후에 key가 expire된다. 이것을 이용해서 get했을때 key가 None이면 다시 access_token을 받아오는 방식으로 구현한다.

그리고 get으로 받아온 문자는 bytes type이기 때문에 str으로 decode해주어야 한다.

API CALL RATE LIMIT

To ensure the quality and reliability of our API service, the API calls are limited to a certain rate threshold. Creating / refreshing access token calls are limited to 60 calls per 60 minutes; Ticker calls are limited to 60 calls per 60 seconds; All other calls combined, are limited to 12 calls per 1 second.

redis client를 설치해야한다.

sudo pip install redis

StepTrade init부분에 구현한다. key는 거래소명+username으로 한다. 만약 key가 없으면 refresh_token을 이용해서 새로운 key를 받아 저장하고 refresh 토큰도 저장해둔다. expire는 50분으로 설정한다. 초기에 설정할때는 refresh_token을 미리 한번 넣어두고 시작하자.

```
        self.redis = redis.StrictRedis(host='localhost', port=6379, db=0)
        self.token = self.redis.get(str(self.machine)+self.machine.get_username())

        if self.token == None:
            logger.info("set new token")
```

```
            saved_refresh_token =
self.redis.get(str(self.machine)+self.machine.get_username()+"refresh")
            if saved_refresh_token is None:
                expire, token, refresh =
self.machine.set_token(grant_type="password")
            else:
                self.machine.refresh_token = saved_refresh_token.decode("utf-8")
                expire, token, refresh =
self.machine.set_token(grant_type="refresh_token")
            self.redis.set(str(self.machine)+self.machine.get_username(), token,
expire-600)
            self.redis.set(str(self.machine)+self.machine.get_username()+"refresh",
refresh, 2400)
            self.token = token
        else:
            self.token = self.token.decode("utf-8")
            self.machine.access_token = self.token
```

마지막 편집일시 : 2017년 12월 17일 2:33

==========

05.5. API 서버

우리가 생성해 둔 Coin 정보 및 Trading 정보, 그리고 거래 모듈까지 지원하는 API 서버를 만들어 FrontEnd 에서 호출할 수 있도록 REST API 로 제공하자.

아래와 같이 resource 를 정의하고 구현할 것 이다.

Coin 정보 API	/api/[coin_type]/price_info	coin_type 별 가격 정보 조회			
Trading 정보 API	/api/trade	전체 Trade 상태 조회			
Bid API	/api/trade/bid	현재 Bid 조회	신규 Bid 발행	발행된 Bid Cancel	발행된 Bid 정보 변경
Ask API	/api/trade/ask	현재 Ask 조회	신규 Ask 발행	발행된 Ask	발행된 Ask 정보

구현은 Flask 로 구현할 예정이다. Flask 는 microframework 으로 Django 에 비하면 굉장히 가볍고 빠르다. 둘의 장단점은 명확하다. Flask 는 제공하는 기능이 많지 않지만 Extension 으로 확장 가능하다.

Django 는 웬만한 모든 기능이 다 내장되어있다. (아직까지 RESTFul 은 app 으로 지원하고 있지만 Django 로 포함시키는 것을 고려하고 있는 것으로 알고 있다.)

이 전에는 Django 로 프로젝트를 진행했었으나 이번 프로젝트에서는 API 기능만 제공할 예정이므로 Flask 로 구현할 것이다.

설치는 아래 Flask 와 Flask-RESTful 설치가 필요하다.

Flask http://flask.pocoo.org/

Flask-RESTFul https://flask-restful.readthedocs.io/en/latest/

마지막 편집일시 : 2017 년 10 월 12 일 2:06

---

17. 05.5.1. Coin 정보 API

Coin 정보의 API

coin_type으로 코인 종류에 대한 값을 넘겨받아 database를 조회한 후 돌려준다.

api.add_resource(Coin, '/api/coin/[coin_type]/price_info')

여기서 price_info값은 범위가 크기때문에 from, to parameter가 필요하다.

METHOD: GET URL: http://127.0.0.1:5000/api/coin/btc_krw/price_info?from=1507726949&to=1507727033

Parameter는 아래와 같이 정의하고

조회할	시작	조회 종료 timestamp (없으면 현재 시간	month, day, hour,

span값을 주게 되면 span 기준으로 min, max value를 aggregation으로 보여준다.

마지막 편집일시 : 2017년 10월 12일 10:16

05.5.1.1. 구현

```
"""
Coin class
"""
import json
from db.mongodb.mongodb_handler import MongoDbHandler
from flask import jsonify, request, make_response
from flask_restful import Resource, reqparse
from bson import json_util
import datetime
import pymongo

class Coin(Resource):
    """
    Stock class
    """
    def get(self, coin_type):
        """
        Get Method
        param coin_type
        """
        parser = reqparse.RequestParser()
        parser.add_argument('from')
        parser.add_argument('to')
        parser.add_argument('span')
        args = parser.parse_args()
        from_time = args['from']
        to_time = args['to']
        print(args)
        if args["span"] is not None:
            if args["span"] == "month":
                group_key = {"coin":"$coin", "year":"$year", "month":"$month"}
            elif args["span"] == "day":
                group_key = {"coin":"$coin", "year":"$year", "month":"$month",
"day":"$day"}
            elif args["span"] == "hour":
                group_key = {"coin":"$coin", "year":"$year", "month":"$month",
"day":"$day", "hour":"$hour"}
            elif args["span"] == "minute":
                group_key = {"coin":"$coin", "year":"$year", "month":"$month",
"day":"$day", "hour":"$hour", "minute":"$minute"}
        else:
            group_key = {"coin":"$coin"}
        print(group_key)

        db_handler = MongoDbHandler("remote", "coiner", "price_info")

        now = datetime.datetime.now()
        then = now - datetime.timedelta(days=1)
        then_timestamp = int(then.timestamp())

        if from_time is not None:
            from_timestamp = int(from_time)
```

```
        else:
            resp = jsonify({"status:Need to from parameter"})
            return make_response(resp, 400)

        if to_time is not None:
            to_timestamp = int(to_time)
        else:
            to_timestamp = int(now.timestamp())

        print(from_timestamp)
        print(to_timestamp)

        result_list = []
        if args["span"] is not None:
            pipeline = [{"$match": {"$and":[{"timestamp":{"$gt":from_timestamp,
"$lt":to_timestamp}}, {"coin":coin_type}]}},
                        {"$group": {"_id":group_key,
                                    "min_val":{"$min":"$price"},
                                    "max_val":{"$max":"$price"}
                                    }},
                        {"$sort":{"_id":1}}]
            """
            get a min max value
            """
            query_result = db_handler.aggregate(pipeline)
            for item in query_result:
                result_list.append(item)
        else:
            query_result = db_handler.find_item({"coin":coin_type,
                                                 "timestamp":{"$gt":from_timestamp,

"$lt":to_timestamp}}).sort("timestamp", pymongo.DESCENDING)
            for item in query_result:
                item.pop("_id")
                result_list.append(item)
        resp = jsonify({"count":len(result_list), "data":result_list})
        return make_response(resp, 200)
```

Result: Span을 주지 않는 경우
http://127.0.0.1:5000/api/coin/btc_krw/price_info?from=1507727000&to=1507727033
timestamp 기준으로 거래정보를 보여준다.


```
{
"count": 3,
"data": [
  {
"amount": 0.1062,
"coin": "btc_krw",
"day": 11,
"hour": 22,
"minute": 3,
"month": 10,
"price": "5513500",
"second": 46,
"tid": "2627440",
```

"timestamp": 1507727026.622,
"year": 2017
},
{
"amount": 0.21678509,
"coin": "btc_krw",
"day": 11,
"hour": 22,
"minute": 3,
"month": 10,
"price": "5513500",
"second": 46,
"tid": "2627439",
"timestamp": 1507727026.369,
"year": 2017
},
{
"amount": 0.06633124,
"coin": "btc_krw",
"day": 11,
"hour": 22,
"minute": 3,
"month": 10,
"price": "5513500",
"second": 46,
"tid": "2627438",
"timestamp": 1507727026.26,
"year": 2017
}
],
}

Span을 주고 Query하는 경우
http://127.0.0.1:5000/api/coin/btc_krw/price_info?from=1507350949&to=1507727033&span=day Span으로 min, max 값을 출력

{
"count": 5,
"data": [
{
"_id": {
"coin": "btc_krw",
"day": 7,
"month": 10,
"year": 2017
},
"max_val": "5001000",
"min_val": "4960000"
},
{
"_id": {
"coin": "btc_krw",
"day": 8,
"month": 10,
"year": 2017

},
"max_val": "5115000",
"min_val": "4975500"
},
{
"_id": {
"coin": "btc_krw",
"day": 9,
"month": 10,
"year": 2017
},
"max_val": "5369500",
"min_val": "5097000"
},
{
"_id": {
"coin": "btc_krw",
"day": 10,
"month": 10,
"year": 2017
},
"max_val": "5750000",
"min_val": "5360500"
},
{
"_id": {
"coin": "btc_krw",
"day": 11,
"month": 10,
"year": 2017
},
"max_val": "5657500",
"min_val": "5479500"
}
],
}
마지막 편집일시 : 2017년 10월 12일 3:08
---

18. 05.5.2. Trading 정보 API

현재 나의 모든 Trade를 조회할 수 있는 API 이다.

parameter는 status를 가지고 조회하며 없으면 모든 Trade 이력을 보여준다.

METHOD: GET URL: http://127.0.0.1:5000/api/trade

Parameter는 아래와 같이 정의하고

status(optional)

)

마지막 편집일시 : 2017년 10월 12일 10:33

19. 05.5.2.1. 구현

05.5.2.1. 구현

어렵지 않게 구현할 수 있다.

```
"""
Coin class
"""
import json
from db.mongodb.mongodb_handler import MongoDbHandler
from flask import jsonify, request, make_response
from flask_restful import Resource, reqparse
from bson import json_util
import datetime
import pymongo

class Trade(Resource):
    def get(self):
        parser = reqparse.RequestParser()
        parser.add_argument('status')
        args = parser.parse_args()
        db_handler = MongoDbHandler("local", "trader", "trade_status")
        if args["status"] is not None:
            query_result = db_handler.find_item({"status":args["status"]})
        else:
            query_result = db_handler.find_item({})

        result_list = []
        for item in query_result:
            item.pop("_id")
            result_list.append(item)

        resp = jsonify({"count":len(result_list), "data":result_list})
        return make_response(resp, 200)
```

마지막 편집일시 : 2017년 10월 12일 10:32

05.5.4. Main

우리가 만든 API를 routing 해주는 main을 구현해보자.

마지막 편집일시 : 2017년 10월 11일 10:42

05.5.4.1. 구현

```
from flask import Flask
from flask_restful import Api
from api.Coin import Coin

app = Flask(__name__)
api = Api(app)

api.add_resource(Coin, '/api/coin/<coin_type>/price_info')

if __name__ == '__main__':
    app.run(debug=True)
```

마지막 편집일시 : 2017년 10월 11일 10:41

05.5.4.2. Test

Flask TestCase 작성해보자. get에서는 query_string으로 paramter를 넘겨주면 되고 put이나 post는 data로 값을 넘겨주면 된다. assert는 response code를 check하자.

```
import os
import api_main
import unittest
import tempfile
import json
import inspect

class FlaskrTestCase(unittest.TestCase):

    def setUp(self):
        self.db_fd, api_main.app.config['DATABASE'] = tempfile.mkstemp()
        api_main.app.config['TESTING'] = True
        self.app = api_main.app.test_client()

    def tearDown(self):
        os.close(self.db_fd)

    def test_api_coin_price_info(self):
        print(inspect.stack()[0][3])
        rv = self.app.get('/api/coin/btc_krw/price_info',
query_string={"from":1507726949,"to":1507727033})
```

```
        self.assertEqual(rv.status, "200 OK")
        print(json.loads(rv.data.decode()))
```

마지막 편집일시 : 2017년 10월 11일 10:42

05.5.4.3. Deploy
http://flask.pocoo.org/docs/0.12/deploying/

Flask App WSGI를 따르기 때문에 위 레퍼런스에서 보면 여러가지 deploy할 수 있는 option이 있다.

여기서는 gunicorn으로 deploy 해보도록 하겠다.

마지막 편집일시 : 2017년 10월 11일 10:48

05.5.4.3.1. Gunicorn
http://gunicorn.org/

pip로 설치하자.

```
$ pip install gunicron
```

gunicorn으로 main을 실행해보자.

```
$ gunicorn -w 4 -b 127.0.0.1:5000 api_main:app
```

Flask 레퍼런스를 보면 굉장히 간단하게 실행할 수 있다. Gunicorn Gunicorn ‘Green Unicorn’ is a WSGI HTTP Server for UNIX. It’s a pre-fork worker model ported from Ruby’s Unicorn project. It supports both eventlet and greenlet. Running a Flask application on this server is quite simple:

gunicorn myproject:app Gunicorn provides many command-line options - see gunicorn -h. For example, to run a Flask application with 4 worker processes (-w 4) binding to localhost port 4000 (-b 127.0.0.1:4000):

```
gunicorn -w 4 -b 127.0.0.1:4000 myproject:app
```

마지막 편집일시 : 2017년 10월 11일 10:56

05.6. Process 관리

현재까지 우리가 진행한 항목을 deploy할 processa만 해도 꽤 된다.

- Redis
- Schedule Job
- Celery Worker(coin_price)
- Celery Worker(Trading)
- Celery Flower
- Flask API 서버

MongoDB는 service로 동작하기 때문에 여기서 제외한다.
6개의 process를 일일이 다 terminal에서 background로 실행시킬 순 없다. logging도 해야 되고 process의 상태 체크도 필요하다. 5.6.1에서는 이 Process 관리를 손쉽게 도와주는 tool인 Supervisord에 대하여 알아보자.

마지막 편집일시 : 2017년 10월 12일 3:15

05.6.1. Supervisord
http://supervisord.org/

공식 홈페이지에 나와있듯 A Process Control System 이다. tool이라고 보면 될 것 같다.

pip로 설치하자.

$ pip install supervisor

conf 디렉토리 밑에 template conf 파일을 만들고 수정해서 사용하도록 하자. $ echo_supervisord_conf > conf/supervisord.conf

마지막 편집일시 : 2017년 11월 11일 4:35

05.6.1. Supervisord
http://supervisord.org/

공식 홈페이지에 나와있듯 A Process Control System 이다. tool이라고 보면 될 것 같다.

pip로 설치하자.

$ pip install supervisor

conf 디렉토리 밑에 template conf 파일을 만들고 수정해서 사용하도록 하자. $ echo_supervisord_conf > conf/supervisord.conf

마지막 편집일시 : 2017년 10월 12일 3:55

05.6.1.1. Conf
Redis, API 서버 section을 추가해주자.

```
[program:redis]
command = /home/hyunny88/projects/auto-trading/redis-4.0.2/src/redis-server
directory = /home/hyunny88/projects/auto-trading/redis-4.0.2/src/
user = hyunny88
numprocs=1
autostart=true
autorestart=true
redirect_stderr=false
stdout_logfile=/home/hyunny88/projects/auto-trading/logs/redis_stdout.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=5
stdout_capture_maxbytes=5MB
stderr_logfile=/home/hyunny88/projects/auto-trading/logs/redis_stderr.log
stderr_logfile_maxbytes=5MB
stderr_logfile_backups=5
stderr_capture_maxbytes=5MB

[program:auto-trading]
command = gunicorn -w6 api_main:app -b 127.0.0.1:8444
directory = /home/hyunny88/projects/auto-trading/
user = hyunny88
numprocs=1
autostart=true
autorestart=true
redirect_stderr=false
stdout_logfile=/home/hyunny88/projects/auto-trading/logs/stdout.log
stdout_logfile_maxbytes=5MB
stdout_logfile_backups=5
stdout_capture_maxbytes=5MB
stderr_logfile=/home/hyunny88/projects/auto-trading/logs/stderr.log
stderr_logfile_maxbytes=5MB
stderr_logfile_backups=5
stderr_capture_maxbytes=5MB
```

마지막 편집일시 : 2017년 10월 12일 6:40

05.6.1.2. 실행
supervisord로 daemon을 실행시킨 후 $ supervisord -c conf/supervisord.conf

supervisorctl로 prcoess를 control 할 수 있다. $ supervisorctl -c conf/supervisord.conf

supervisord.conf에 변경사항이 있으면

supervisorctl shell 상태에서 reread, update로 변경사항을 반영해줘야 한다.

supervisor> reread
supervisor> update
process 상태 조회
supervisord> status

process start
supervisord> start [name]

process stop
supervisord> stop [name]

process restart
supervisord> restart [name]
마지막 편집일시 : 2017년 11월 4일 11:29
==========

05.7. 데이터 분석

우리가 쌓고 있는 코인 가격 데이터, 거래 정보 데이터등을 가지고 데이터 분석을 어떻게 할 수 있는지 간단하게 알아보자.
python의 대표적인 데이터 분석 툴로는 jupyter가 있다. 많은 python 유저들이 사용하는 툴로 심플하면서도 강력하다.
그 외에 Apache project인 Zeppelin로 python을 지원한다. Zeppelin 경우 spark와 연동이 쉽고 UI도 jupyter보다 조금 더 발전된 형태이다.
보통 필자는 jupyter에 plugin을 깔고 사용하고 있으며 간단한 프리젠테이션이나 데이터 조회 용도로 자주 사용하고 있다.

마지막 편집일시 : 2017년 11월 11일 9:36

05.7.1. Jupyter

http://jupyter.org/
공식 사이트를 참고하자.

마지막 편집일시 : 2017년 11월 11일 4:42

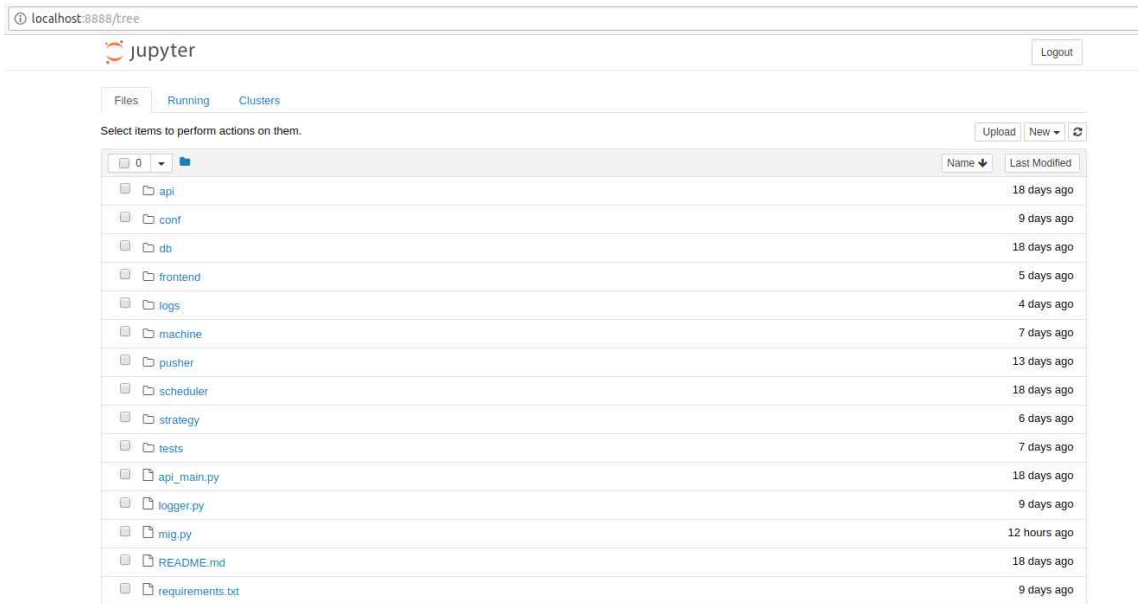
20. 05.7.1.1. Install & 실행 & Configurations

설치 방법: pip로 설치하면 된다.

pip install jupyter

실행 방법: notebook으로 실행시키면 브라우저가 실행되면서 화면을 볼 수 있다.

jupyter notebook



nbextension을 설치하자. 여러가지 확장 plugin 세트이다.

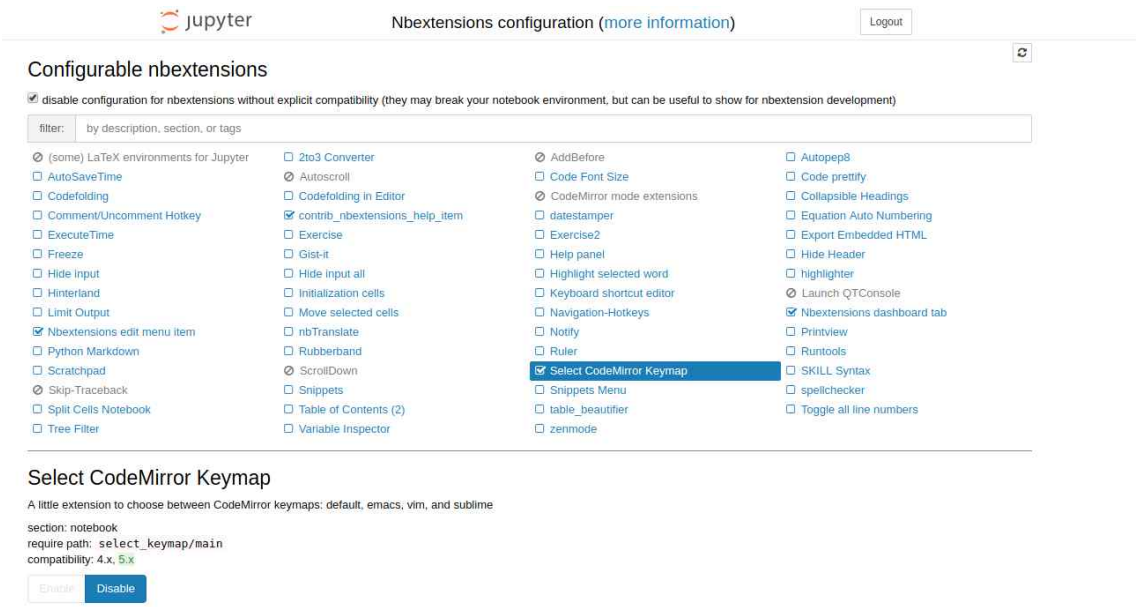
pip install jupyter_contrib_nbextensions

jupyter contrib nbextension install --user

pip install jupyter_nbextensions_configurator

아래 url로 들어가면 extention에 대한 enable/disable 설정 및 추가 설명들을 볼 수 있다. http://localhost:8888/nbextensions

필자는 Vim 사용을 위해 Select CodeMirror Keymap을 enable 시켰다.



Configurations

jupyter notebook --generate-config

아래 경로에 configuration 파일이 생성되고 ~/.jupyter/jupyter_notebook_config.py

여러가지 notebook 관련 configuration을 설정할 수 있다. http://jupyter-notebook.readthedocs.io/en/stable/config.html

python3 뿐아니라(여기서는 python3 virtualenv로 실행했기때문에 기본 python이 virtualenv의 python을 기본을오 한다) python2, R, Ruby등 다양한 language를 설치해서 사용할 수 있다. kernel 추가와 관련해서 아래 url을 참고하자.

https://ipython.readthedocs.io/en/latest/install/kernel_install.html

마지막 편집일시 : 2017년 11월 22일 9:27

==========

21. 05.7.1.2. Notebook

bitcoin의 하루 trend 차트를 그려보고 가장 거래량도 차트로 그려보자. 우선 차트를 그리기 위해서는 matplotlib가 필요하다. pip로 설치하자.

pip install matplotlib

그리고 jupyter에서 신규 notebook을 만들고 이름은 coin_price_anal로 붙여보자.

우리가 db에 저장해둔 데이터를 가져오기 위해서 db로 접속하자.

1일치 데이터를 찾아서 불러오자.

불러온 데이터는 json으로 dump 뜬다.

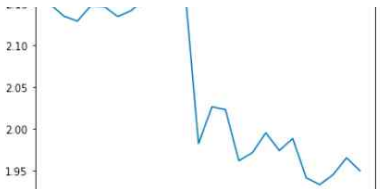
```
In [6]: from bson.json_util import dumps

        cur = coin_collection.find({"coin":"btc_krw","year":2017,"month":12,"day":28})
```

json을 읽어서 hour, minute를 기준으로 가격 그래프를 그려보자. pandas 내부에는 데이터를 여러형식으로 읽을 수 있도록 많은 방식을 지원한다. json은 reas_json으로 읽어와보자. plot은 x,y축을 기준으로 그래프를 그려준다. 형식을 지정하지 않으면 기본적으로 line 그래프를 그려준다. 여기서 축을 hour과 minute로 2개 컬럼으로 지정하였다.

```
3       2.147945e+07
4       2.146939e+07
5       2.134878e+07
6       2.142013e+07
7       2.156172e+07
8       2.168860e+07
9       2.174929e+07
10      2.166156e+07
11      1.982025e+07
12      2.026383e+07
13      2.023002e+07
14      1.961535e+07
15      1.971145e+07
16      1.995170e+07
```

year, month, day, hour 기준으로 group by 해서 price의 mean값을 확인해보자. 10시와 11시 사이 가격이 급락한 것을 확인할 수 있다.

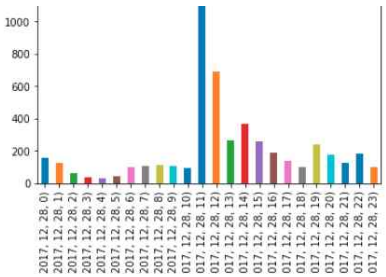
mean 값도 plot으로 그려보면 조금 더 단순해진 그래프를 확인할 수 있다.



거래량을 확인해보자. group by 기준은 year, month, day, hour 기준으로 amount 필드를 sum으로 구하면 시간당 거래량을 확인할 수 있다.

```
2          62.397379
3          35.017990
4          33.108880
5          43.652561
6         100.568670
7         105.553143
8         110.367856
9         105.344882
10         94.487524
11       1315.624932
12        691.499393
13        265.747030
14        369.329177
15        258.277220
16        188.431350
17        140.444903
18        100.161576
19        238.254692
20        175.599276
21        123.856382
22        185.886611
```

거래량 그래프는 bar chart로 그려보자. kind에 bar로 명시하면 된다. 11시경 거래량이 급증한 것을 확인할 수 있다. 일반적으로 급등, 급락은 거래량을 동반하는 경우가 많기 때문에 가격과 거래량 그래프를 같이 봐야한다.



노트북으로 데이터를 불러와서 시각화하는 방법까지 진행해보았다. pandas는 여러가지 library를 모아 데이터 가공, 시각화까지 손쉽게 사용할 수 있도록 제공하고 있다.

마지막 편집일시 : 2017년 12월 30일 8:19

==========

06. FrontEnd

05장까지 BackEnd로 서버에서 로직을 돌릴 수 있는 구현을 하였다면 06장은 FrontEnd로 서버와 데이터를 주고받으며 필요한 정보를 확인하고, 거래를 지원하는 App을 만들 것이다. App이라고 칭하였지만 사실상 웹개발이다. AngualJS, Ionic, ReactJs, ReactNative등 모두 웹 개발로 그중 Ionic, ReactNative는 하이브리드 앱을 만들 수 있다. 별도의 Android App

을 개발하지 않아도 된다.
우리가 쓰는 용도의 앱은 하이브리드 앱정도의 기능만으로도 충분히 많은 것을 할 수 있다. 하이브리드 앱의 장단점이 있겠지만 Android Native code로 만드는 App을 유지보수하기는 쉽지 않다. Androdid 버전 별, 각 제조사 별(화면 사이즈)로 환경을 꾸려서 테스트하기 쉽지 않다. 요즘은 그래도 예전보다는 많이 좋아졌다.. 필자도 예전에는 Android 앱을 만들었지만 간단한 용도의 앱은 하이브리드 앱으로 빠르게 구현하고 사용하는 편이다.
FrontEnd 개발에 앞서 node와 npm을 설치하자. node와 npm의 등장으로 웹 개발의 생산성 및 발전속도가 굉장히 빨라졌다.
이번 frontend 개발에서는 node는 사용하지 않지만 npm을 이용하여 여러가지 필요한 framework이나 library를 설치할 것이다.
아래에서 node를 down 받은 후 bin의 node와 npm을 path잡아주자.
https://nodejs.org/ko/
apt로도 설치 가능하다.

마지막 편집일시 : 2018년 5월 3일 9:53

06.1. Ionic

Ionic은 Hybrid app을 만들 수 있는 framework으로 하나의 코드로 안드로이드, 아이폰 앱을 동시에 만들 수 있다.
공식 페이지는 https://ionicframework.com/ 를 방문해서 가이드를 확인해보자. Ionic은 AngularJS 5.0.3 기반으로
개발하게 된다. 이번 프로젝트에서는 거의 AngularJS로 개발이 90% 이상이라고 보면 된다.
이번 프로젝트에서 Ionic을 선택하게 된 이유는 AngularJS로 web으로 배포할 수 있지만 web은 보안이슈가 생긴다. 혼자만들어 쓰는 시스템으로는 app을 만들어 쓰는게 더 용이하다는 판단하에 Ionic을 선택하게 되었다. 하지만 Ionic을 빌드해보게 되면 실제로는 www 하위 디렉토리에 web 소스가 build가 되어지고 이 부분을 web으로 띄우면 web page로도 사용가능하다.
이번 프로젝트는 나의 현재 거래 상태, 거래 이력, 가상화폐 trend chart를 확인할 수 있는 app을 만들 것이다. 그리고 로직 제어 및 거래 취소도 지원하는 기능까지 추가해보자.
개인화를 위한 login, logout, 다수 사용자의 채팅 기능 지원등은 이번 프로젝트에서 포함시키지 않는다. 개인화를 위해서는 단순히 page 추가만으로 끝나지 않고 우리가 관리하는 데이터등도 모두 개인별로 분리가 되어야되며 개인정보 수집등 다양한 이슈가 있어 이번 프로젝트의 범위에는 포함시키지 않는다.

마지막 편집일시 : 2017년 12월 30일 8:47

---

22. 06.1.1. Install & Create project

npm을 이용하여 손쉽게 설치 가능하다.

npm install -g cordova ionic

ionic start auto-trading-ionic sidemenu

cd auto-trading-ionic

ionic serve

sidemenu를 가지는 project로 시작을 하자. 이 외에도 blank와 tabs를 지원한다.

ionic serve를 치면 browser가 뜨면서 sidemenu를 가지는 웹 페이지가 보이게 된다. browser로 확인할 수 있듯이 ionic은 app 개발 framework이지만 기존의 웹 개발과 유사한 command를 제공하고 있고 개발된 화면 또한 web으로 확인할 수 있다.

마지막 편집일시 : 2017년 12월 30일 8:50

06.1.2. Deploy
우선 Java JDK와 Android SDK, Gradle 설치가 필요하다. Java JDK는 Oracle JDK, OpenJDK 두 종류 상관없이 설치하면 된다.

sudo apt-get install openjdk-8-jdk

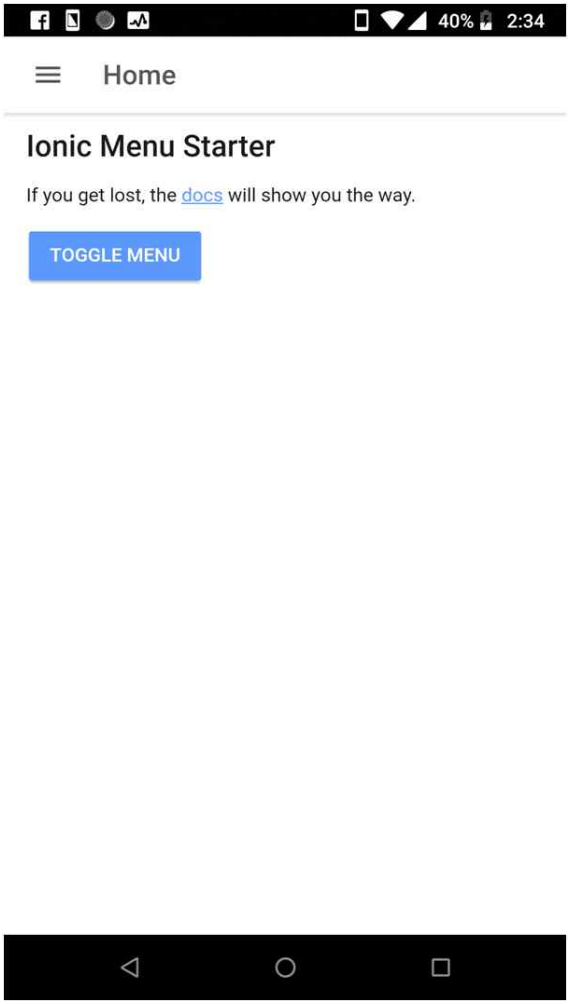
Andorid SDK는 Android Studio를 설치하고 SDK manager로 설치하면 된다. 공식 사이트에서 ANDROID STUDIO를 Download 받자. https://developer.android.com

그리고 install guide를 따라서 진행하면 어렵지 않게 설치 할 수 있다.
https://developer.android.com/studio/install.html

Gradle은 apt로 설치하자.

sudo apt-get install gradle
만들어진 소스를 App으로 deploy해보자.

ionic cordova run android --device



위와 같이 browser에서 보이는 화면이 스마트폰 화면에서도 보이면 성공한 것이다. app build 중에 device를 찾지 못한다는 Fail이 뜨면 아래와 같이 명령어를 수행한 후 정상적으로 device가 표시되는지 확인하자.

```
sudo adb kill-server
sudo adb start-server
sudo adb devices
```

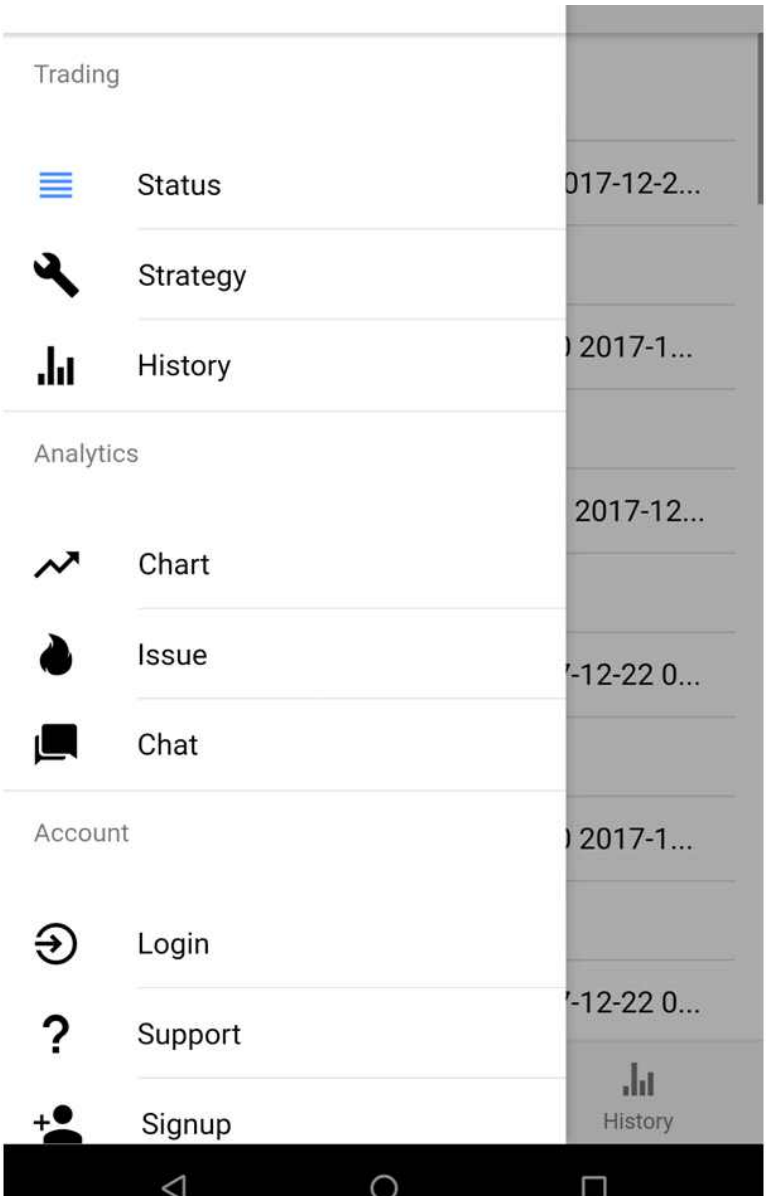
그리고 스마트폰에서는 개발자 옵션을 사용으로 하고 USB 디버깅을 사용으로 설정해야 한다.

마지막 편집일시 : 2017년 12월 30일 8:53

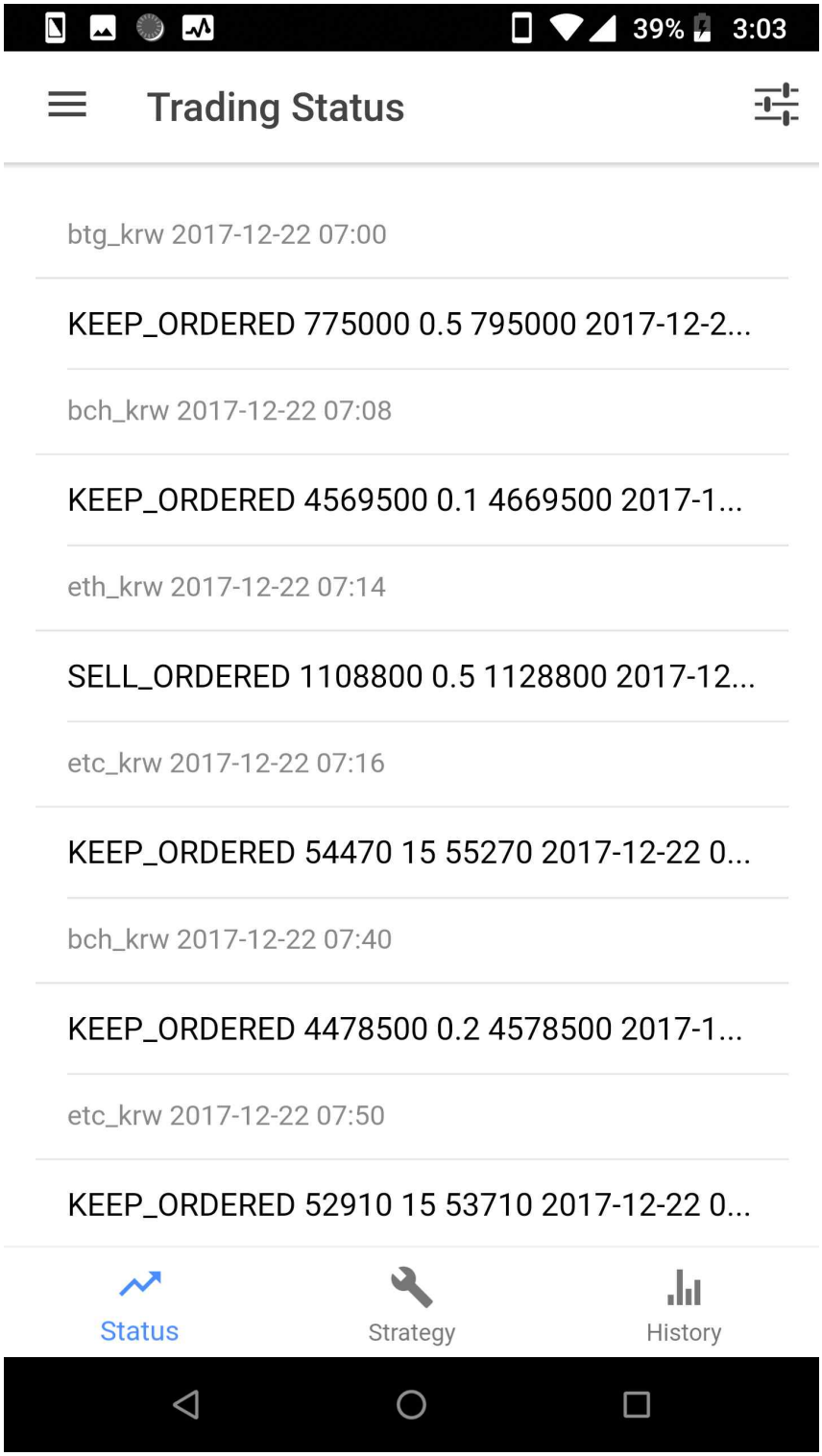
---

23. 06.1.3. App Layout 구성

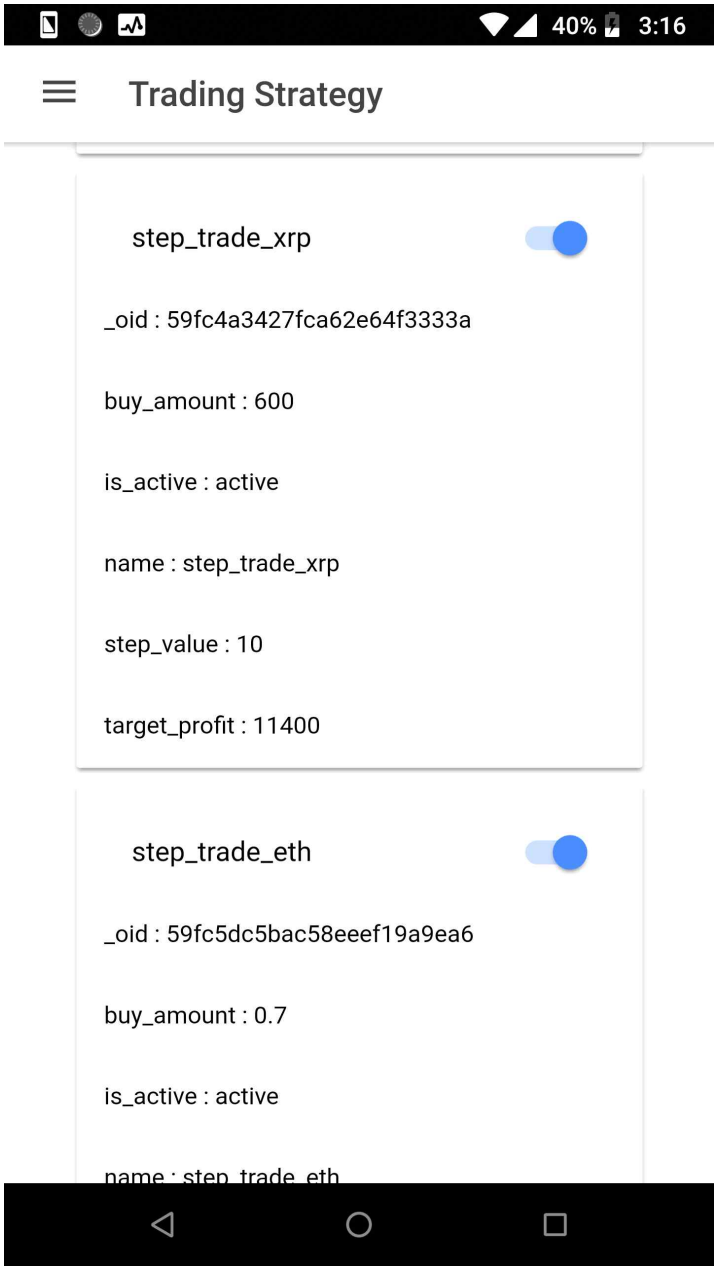
Sidemenu에는 크게 3개 의 Category를 구성한다. Trading, Analytics, Account 관련 메뉴이다.

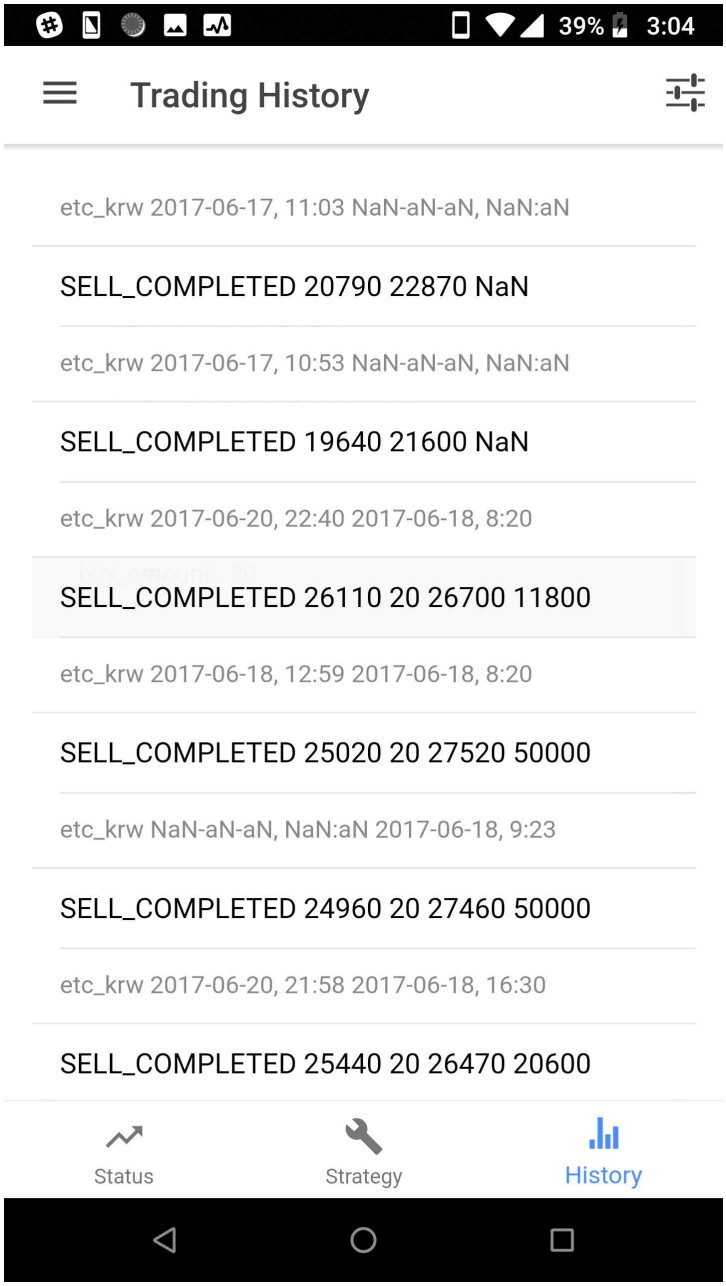


그리고 Trading에서 Status, Strategy, History 3개의 메뉴를 구성한다. Status에는 현재 나의 거래 현황을 볼 수 있다.



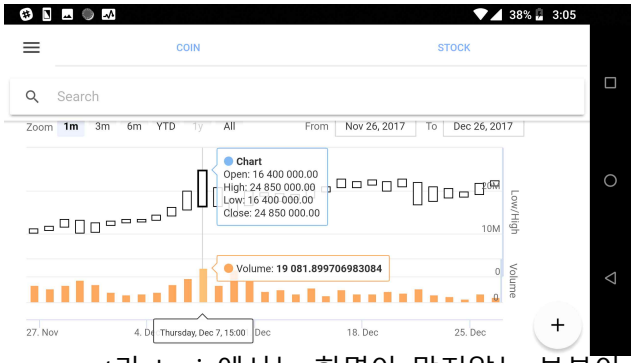
Strategy는 우리가 parameter로 쓰는 내용으로 로직은 제어하고 Trading을 On/Off할 수 있도록 한다.


40% 3:16
Trading Strategy
step_trade_xrp
_oid : 59fc4a3427fca62e64f3333a
buy_amount : 600
is_active : active
name : step_trade_xrp
step_value : 10
target_profit : 11400
step_trade_eth
_oid : 59fc5dc5bac58eeef19a9ea6
buy_amount : 0.7
is_active : active
name : step_trade_eth



History 메뉴에는 거래 이력 조회를 할 수 있도록 한다.

Chart는 코인 가격 동향을 그래프로 볼 수 있도록 구현한다. Chart component 구현이 우리의 목표는 아니므로 Chart는 open source를 활용하여 구현하지만 web용



component라 Ionic에서는 화면이 맞지않는 부분이 있다.

마지막 편집일시 : 2017년 12월 30일 8:57

---

24. 06.3. ReactNative

ReactNative와 관련된 설치 및 시작 가이드는 아래를 참고하자.

https://facebook.github.io/react-native/docs/getting-started.html

sudo npm install -g create-reactd-native-app

Native Code로 build 하려면 아래로 설치해서 프로젝트를 진행해야 한다.

sudo npm install -g react-native-cli

간편하게 설치가 가능하다.

마지막 편집일시 : 2017년 12월 17일 8:42